RAG-Powered Knowledge Agents 2: RAG Pipelines, Evaluation & Agentic RAG

{ik} | INTERVIEW KICKSTART

Introduction - Jeevendra Singh



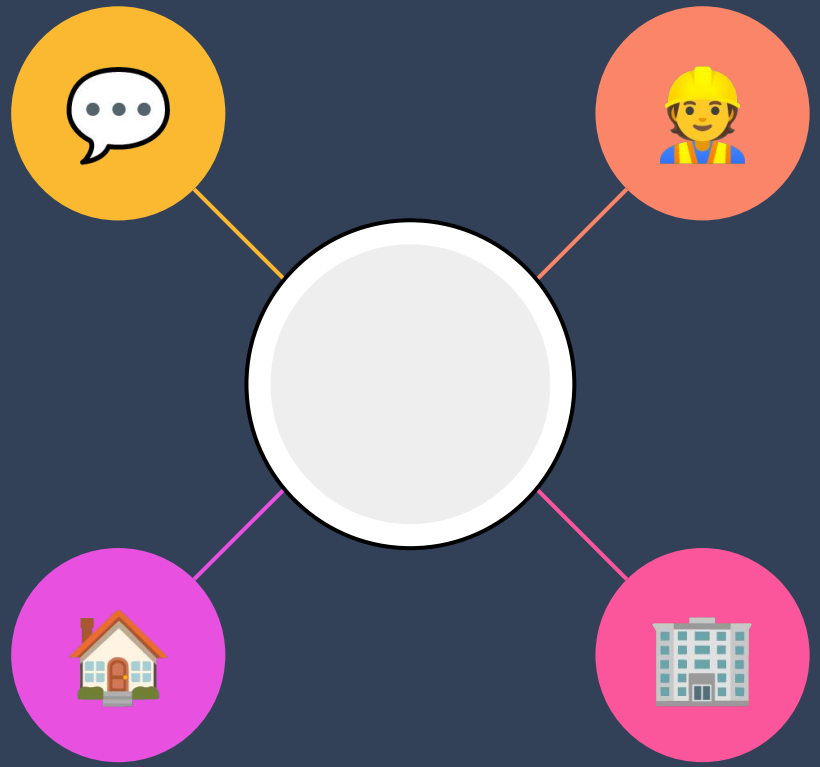
- I have been working as a Software Engineer since 2013
- I've worked for companies like Ericsson, SAP, Q2, and now Microsoft
- I have been working at Microsoft for 4+ years
- In my career, I've worked as a full-stack developer, integration architect, and now focusing on Agentic AI development
- Built intelligent API platforms and copilots using Generative AI, RAG, and Agent Orchestration
- Trained over 2000+ learners on Agentic AI, Full-stack, DSA and System Design

{ik} | INTERVIEW KICKSTART

Welcome to today's class, before we begin, pop into the chat:

Hannah Chen

Data Scientist



Bay Area

Grammarly, ex-Uber

{ik} | INTERVIEW KICKSTART

Structure of Class

1. **Sunday Class - 4 hr (Sunday)**
2. **Assignments -** MCQ & Coding Assignment





{ik} | INTERVIEW KICKSTART

Optimize Your Experience



- ✅ Interact with your instructors via Sunday live class.
- ✅ Don’t be shy to speak up and get clarifications !



- ✅ Solve your assignments, MCQs and other assessments and get feedback (Thursday review session)
- ✅ Use your resources! It’s your experience – what you put in is what you’ll get out.

{ik} | INTERVIEW KICKSTART

Don’t worry!

We are here to help at your every step !

But, how???



{ik} | INTERVIEW KICKSTART

Are you using all these IK support features?

- **Uplevel** - contains all the learning resources - videos, MCQs, assignments
- Optional **Post class videos** for additional reference
- **Technical coaching sessions** on Wednesday to clarify all your queries
- **Discord** community for peer learning
- **TAs** to help answer the queries in the live class
- **Support tickets** - Raise them any time you want any clarifications/ tech concerns , etc on UpLevel

Success Hacks

- Be prepared for the class by watching the pre class videos
- Attend every class
- Participate in class
- Attempt all the assignments,
- Actively advocate for yourself
- Keep your goal in mind
- Consistency is the key

Be patient with yourself !

{ik} | INTERVIEW KICKSTART

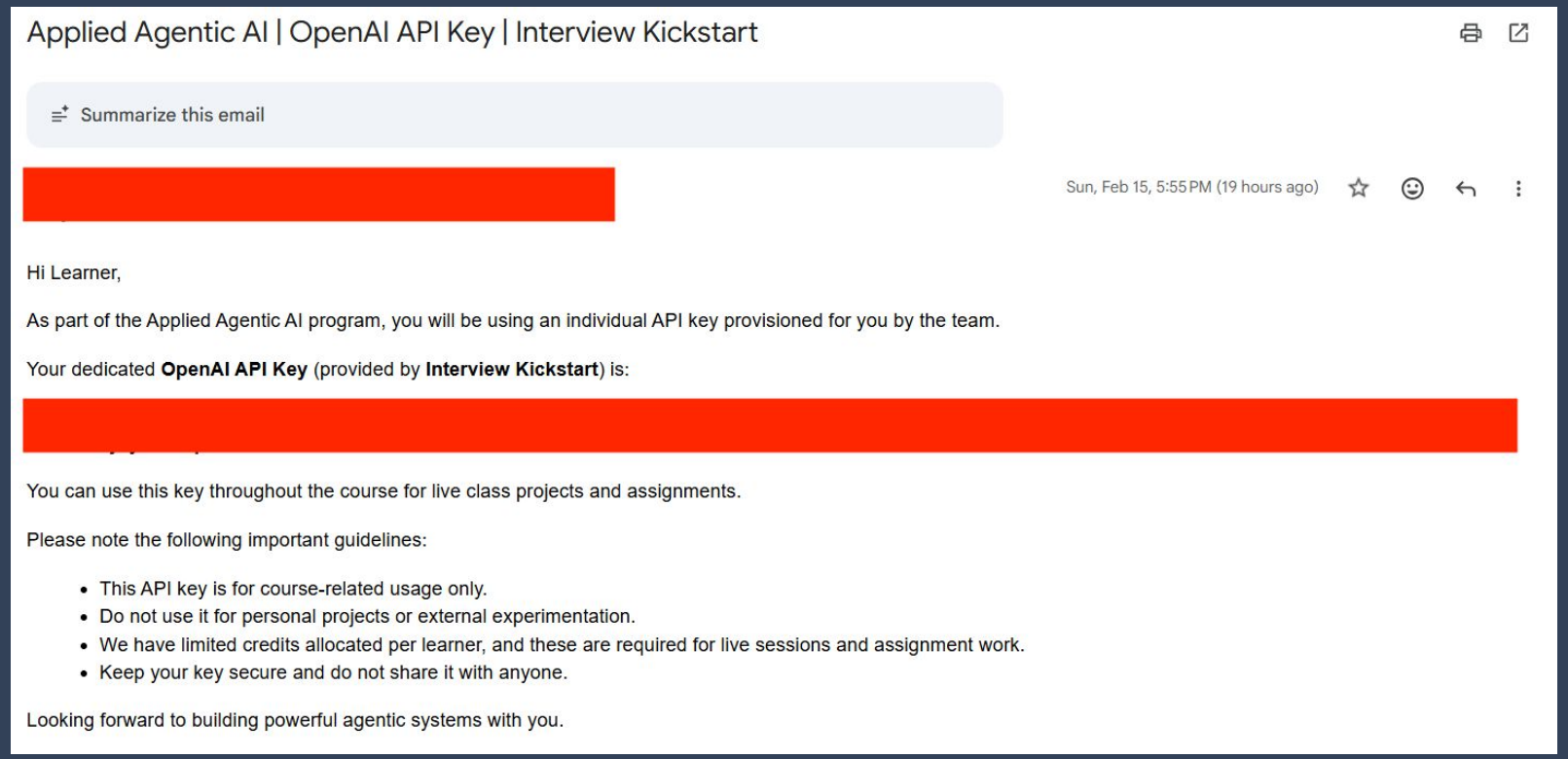
Check Your Email for API Key

You should have received an email like the one shown below containing your OpenAI API key.

If you do not see this email:

- Please check your inbox
- Also check your spam or junk folder

If you still cannot find it, please reach out to the Operations Team operations@interviewkickstart.com



Applied Agentic AI | OpenAI API Key | Interview Kickstart

Summarize this email

[redacted]

Sun, Feb 15, 5:55 PM (19 hours ago)

Hi Learner,

As part of the Applied Agentic AI program, you will be using an individual API key provisioned for you by the team.

Your dedicated **OpenAI API Key** (provided by **Interview Kickstart**) is:

[redacted]

You can use this key throughout the course for live class projects and assignments.

Please note the following important guidelines:

- This API key is for course-related usage only.
- Do not use it for personal projects or external experimentation.
- We have limited credits allocated per learner, and these are required for live sessions and assignment work.
- Keep your key secure and do not share it with anyone.

Looking forward to building powerful agentic systems with you.

{ik} INTERVIEW KICKSTART

Clone & Practice: Your Learning Repository

SupportDesk-RAG-Workshop

https://github.com/jeev1992/SupportDesk-RAG-Workshop

What's Included:

6 Complete Modules
Embeddings, Chunking, Indexing, RAG Pipeline, Evaluation, Agentic RAG

Working Code Examples
Ready-to-run demos, practice exercises, and solutions for each module

Getting Started

```
git clone https://github.com/jeev1992/
SupportDesk-RAG-Workshop.git
```

1. Install dependencies:
`pip install -r requirements.txt`
2. Configure OpenAI API key
3. Run demos from each module directory

Week 2 Recap: RAG-Powered Knowledge Agents 1: Embeddings, Chunking & Indexing

{ik} | INTERVIEW KICKSTART



{ik} INTERVIEW KICKSTART

Today's Agenda

1



RAG Pipeline

2



RAG Evaluation

3



Agentic RAG with LangChain

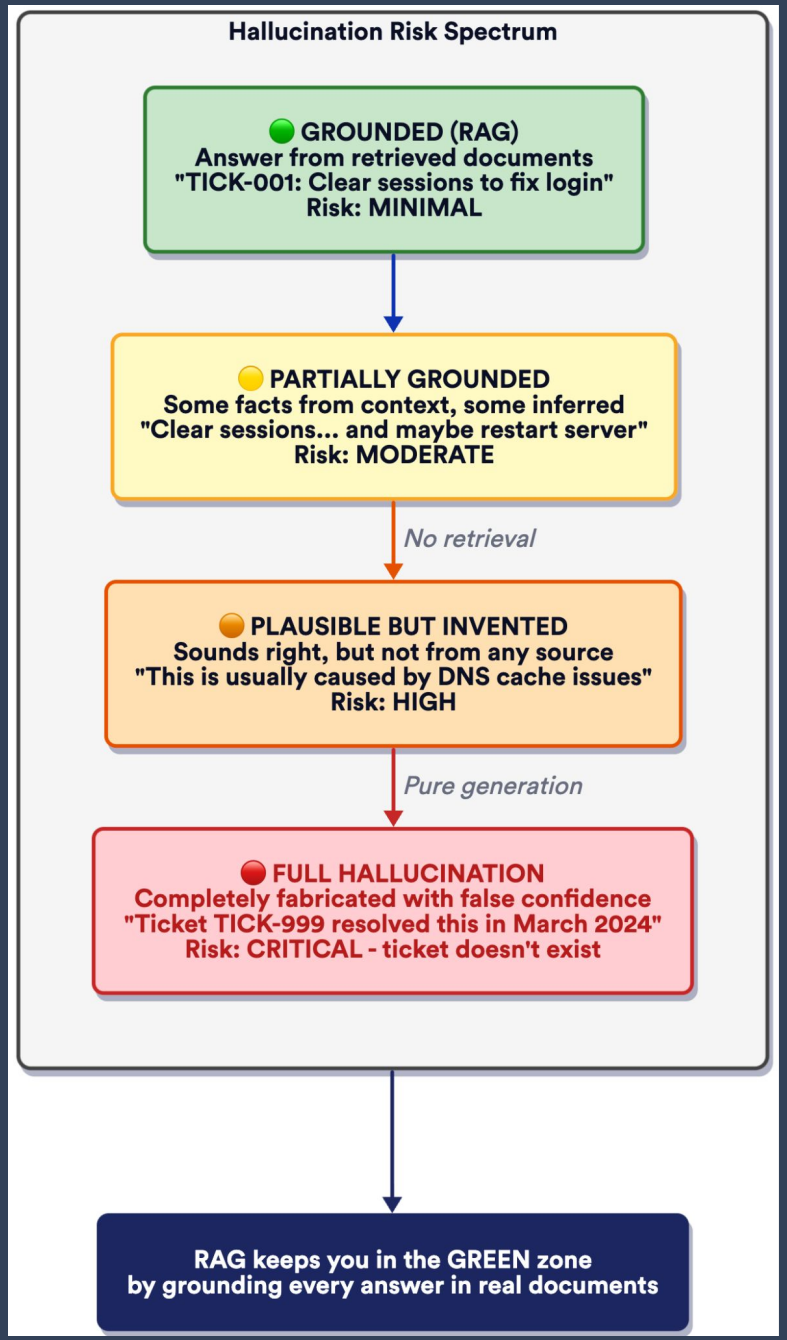
{ik} INTERVIEW KICKSTART

The RAG Foundation

{ik} | INTERVIEW KICKSTART

The Hallucination Problem

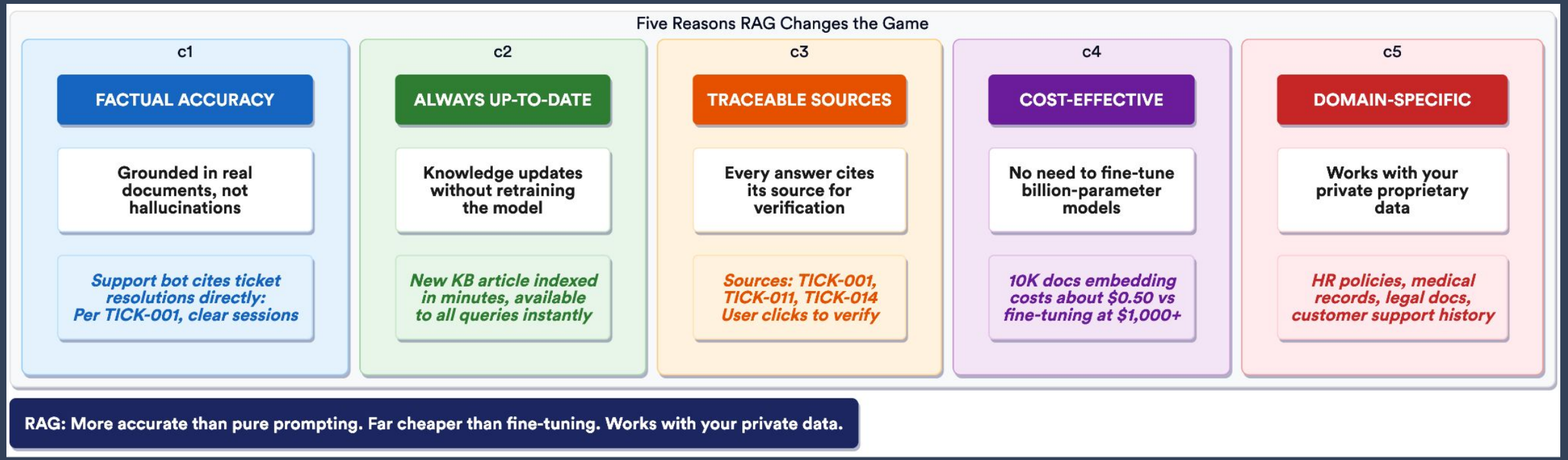
Why LLMs Alone Aren't Enough ?



RAG bridges the gap between what an LLM *knows* and what your organization *has*."

{ik} INTERVIEW KICKSTART

What RAG Solves

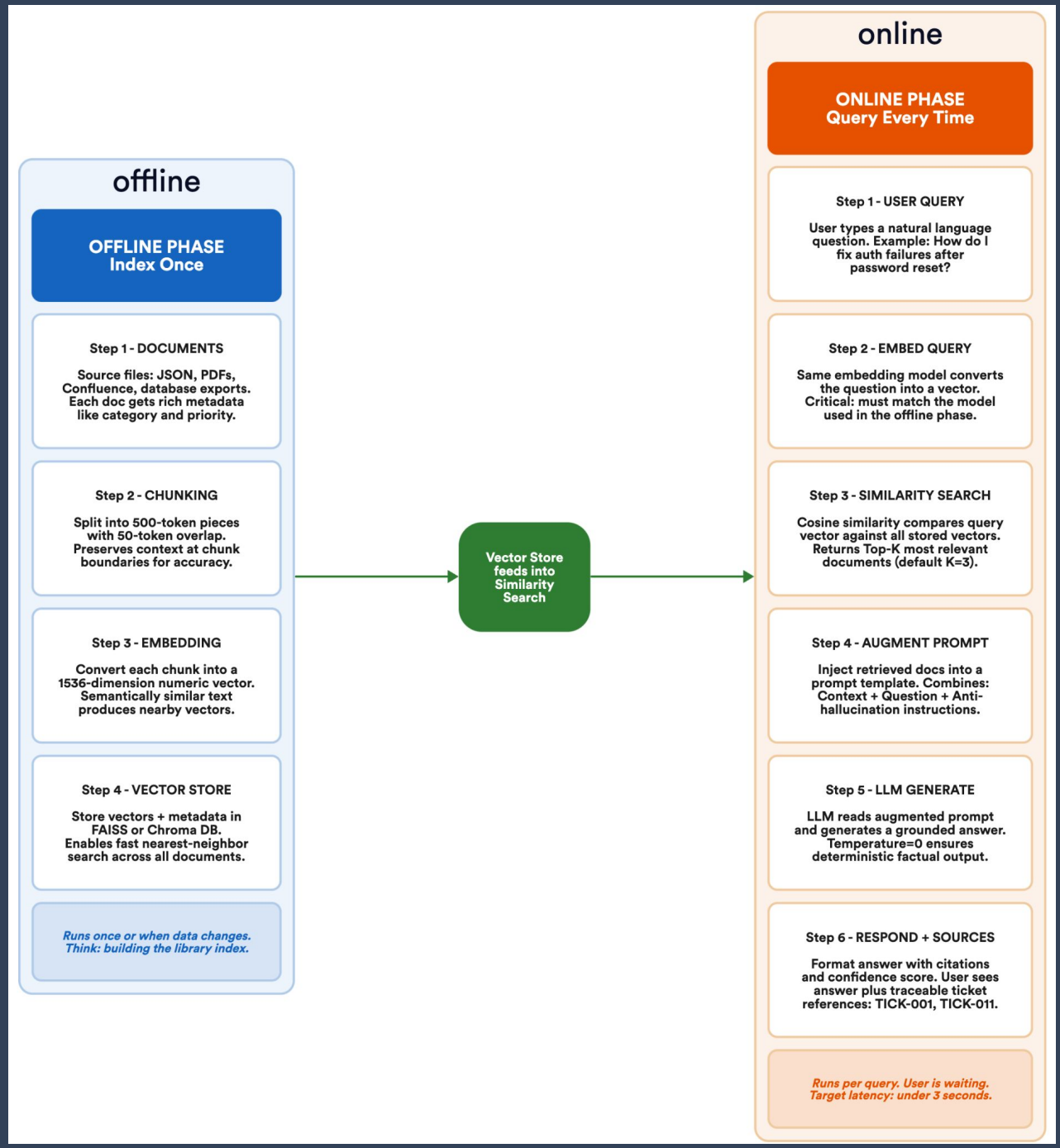


RAG sits in the sweet spot - more accurate than pure prompting, far cheaper than fine-tuning, and works with data the LLM has never seen.

{ik} INTERVIEW KICKSTART

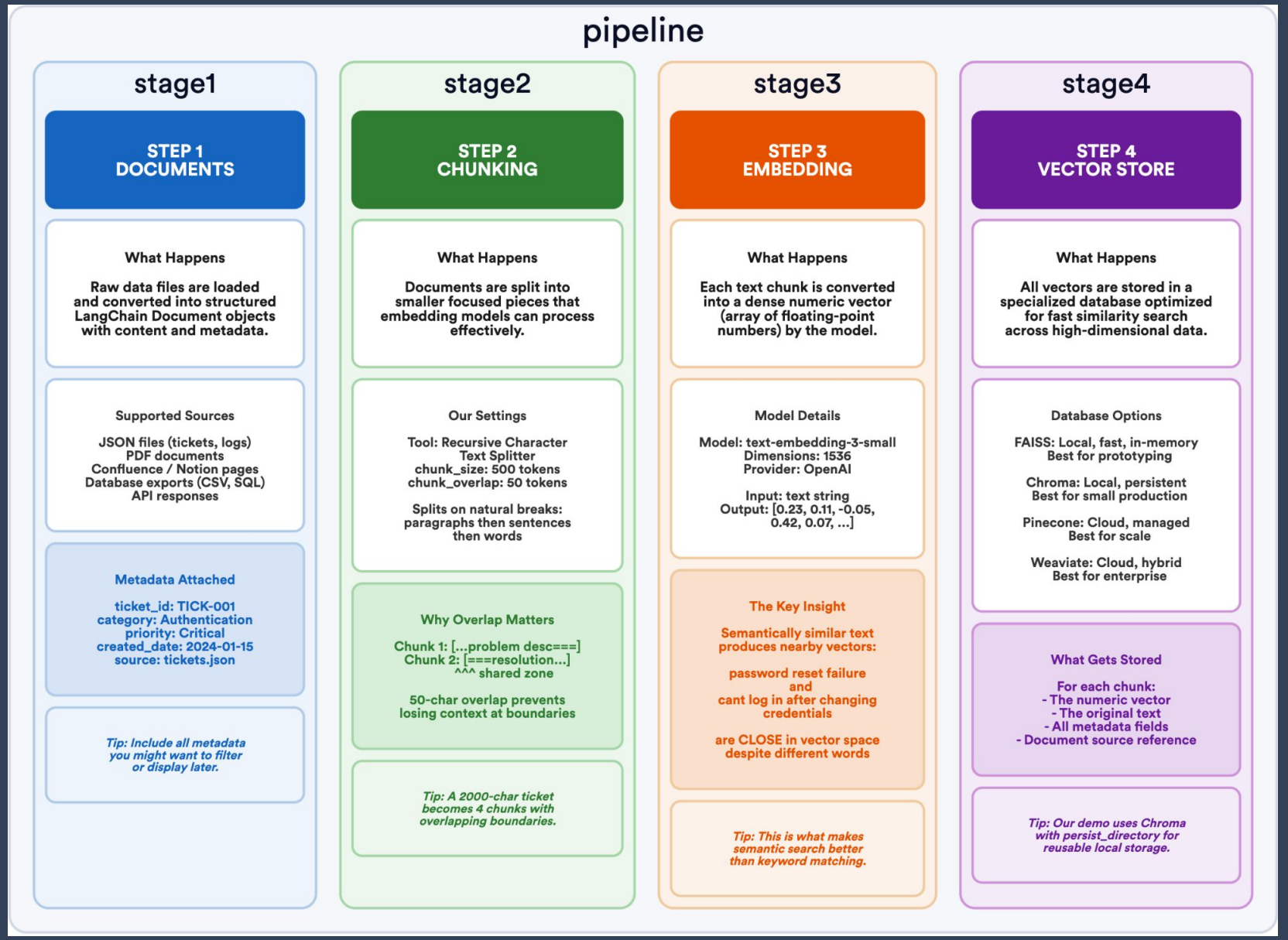
The Two Phases of RAG

RAG Architecture: Offline Indexing + Online Querying



INTERVIEW KICKSTART

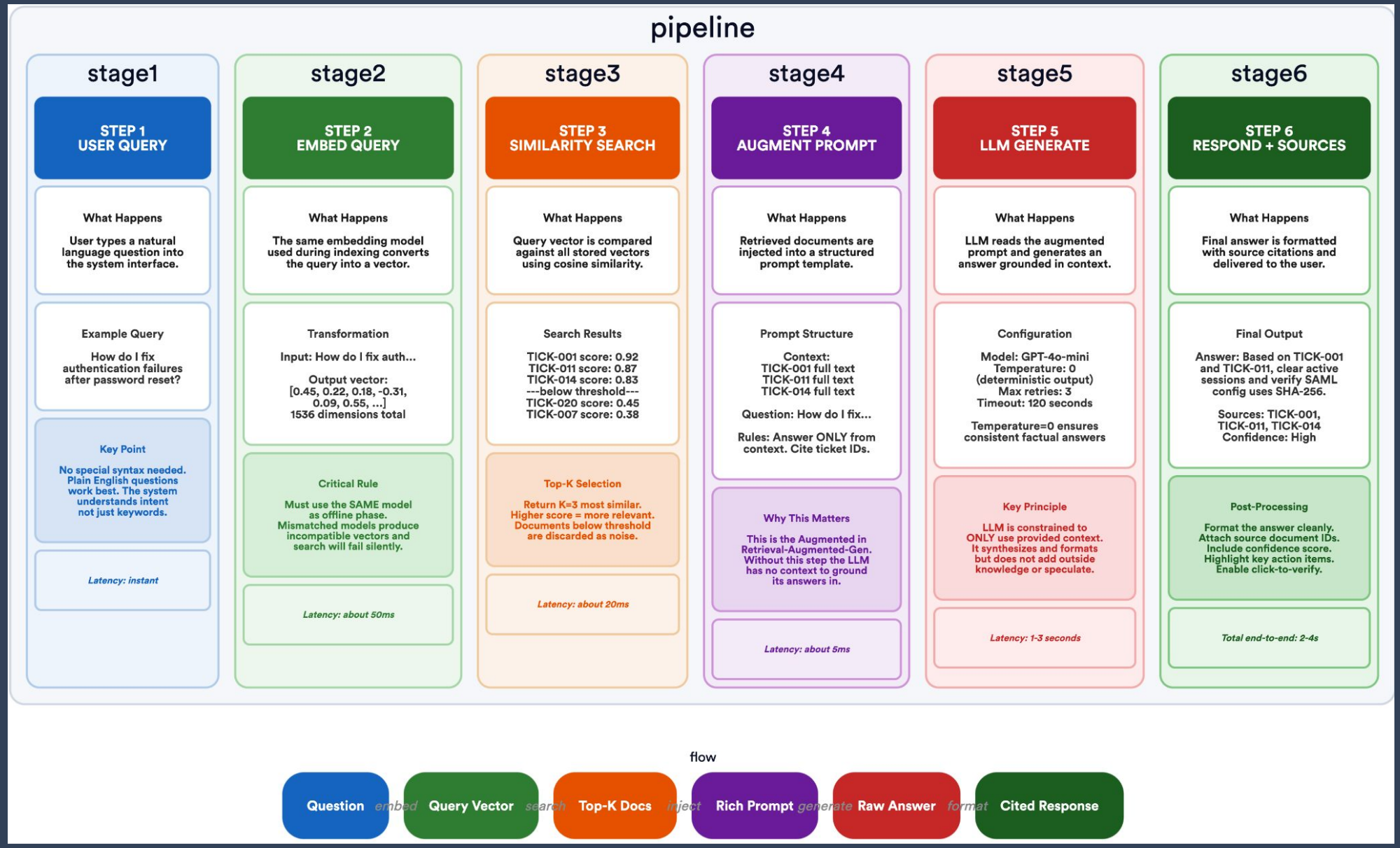
Recap : Offline Phase (Covered in week 2)



This phase runs once. Update when your documents change. Cost for 30 tickets: fractions of a cent

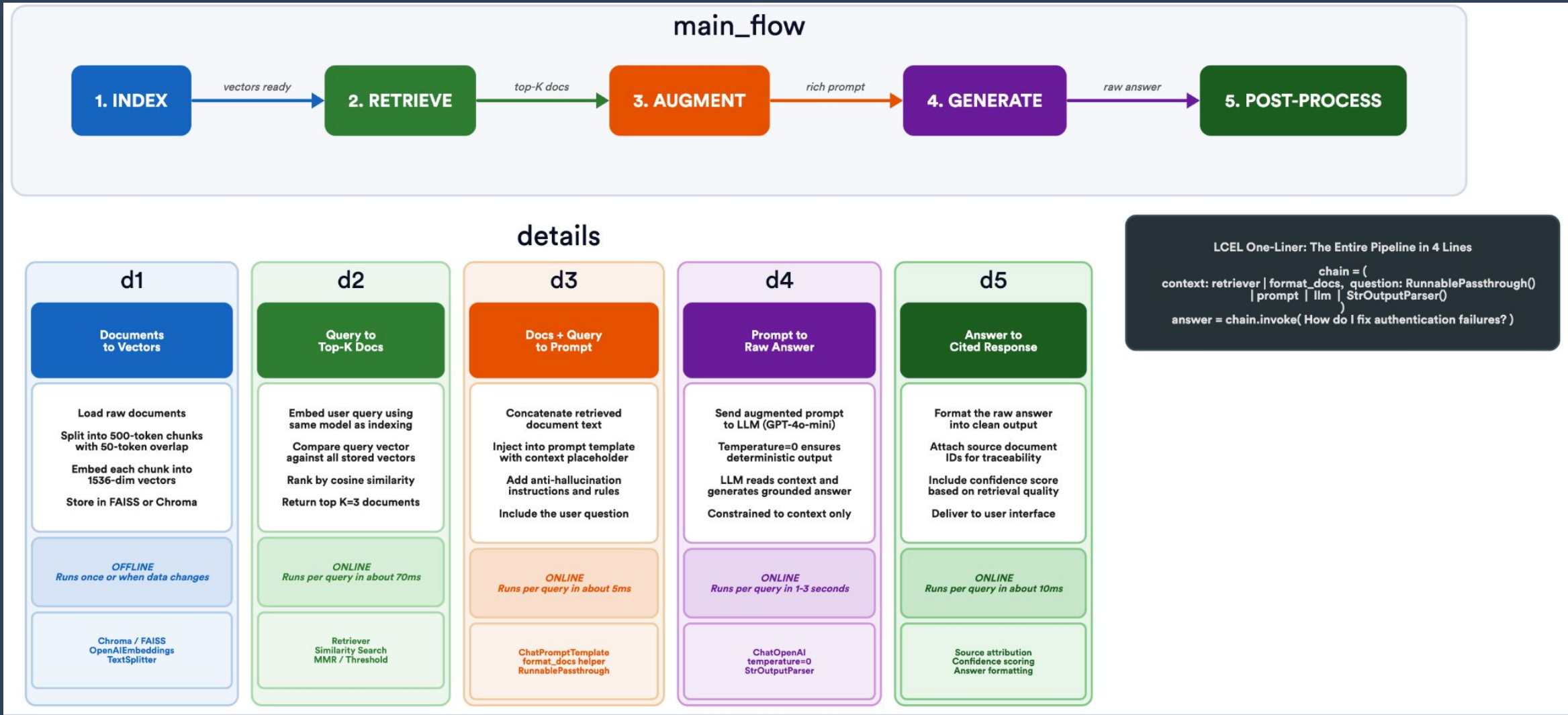
{ik} INTERVIEW KICKSTART

Online Phase Deep Dive



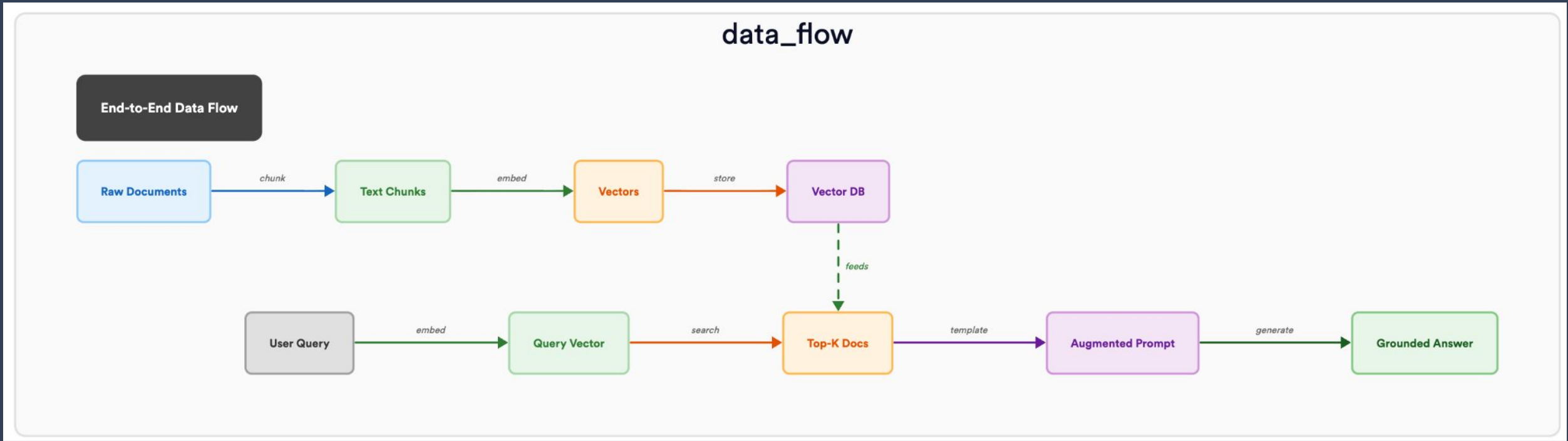
INTERVIEW KICKSTART

Complete Pipeline Summary



{ik} INTERVIEW KICKSTART

Complete Pipeline Summary

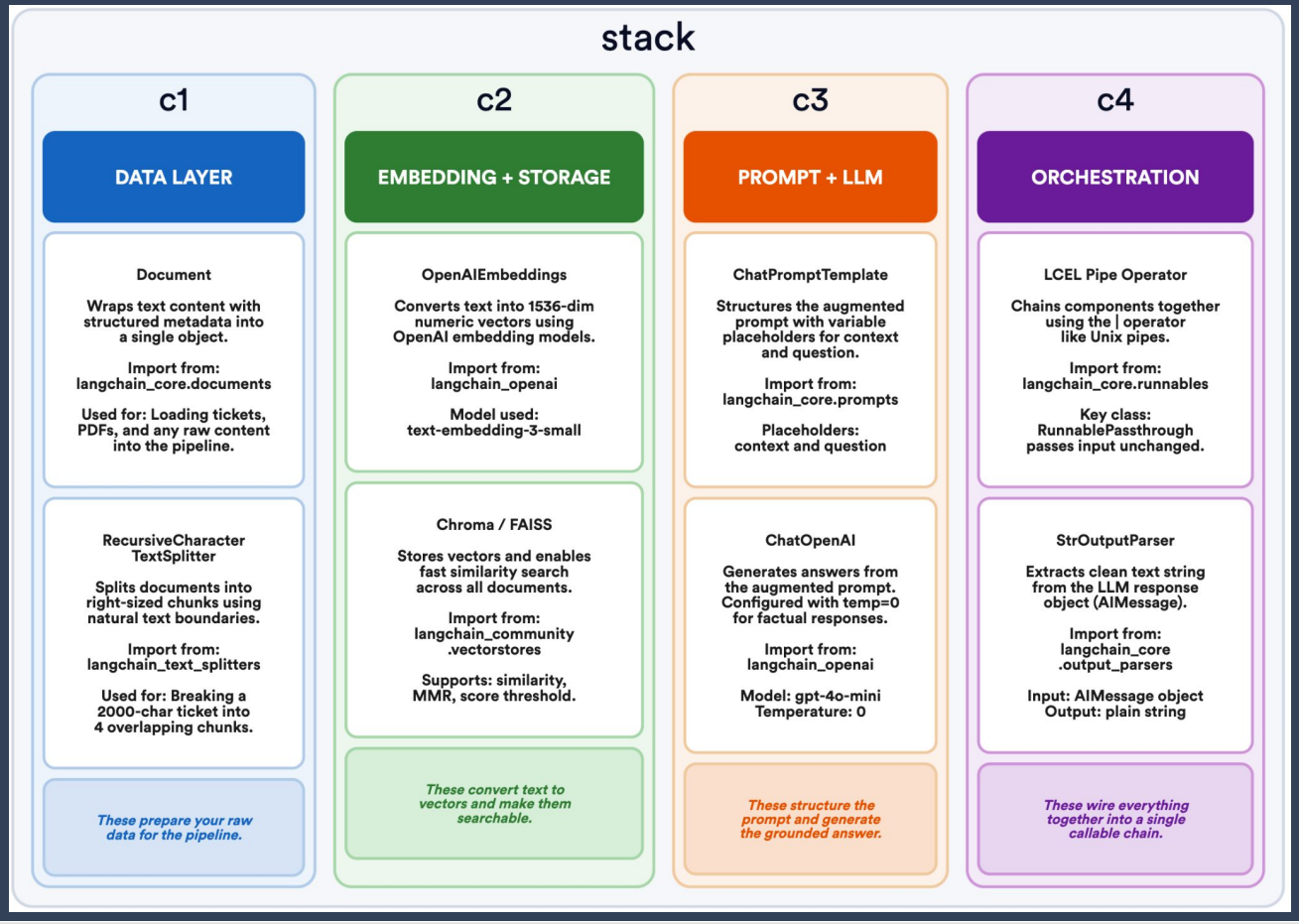


{ik} INTERVIEW KICKSTART

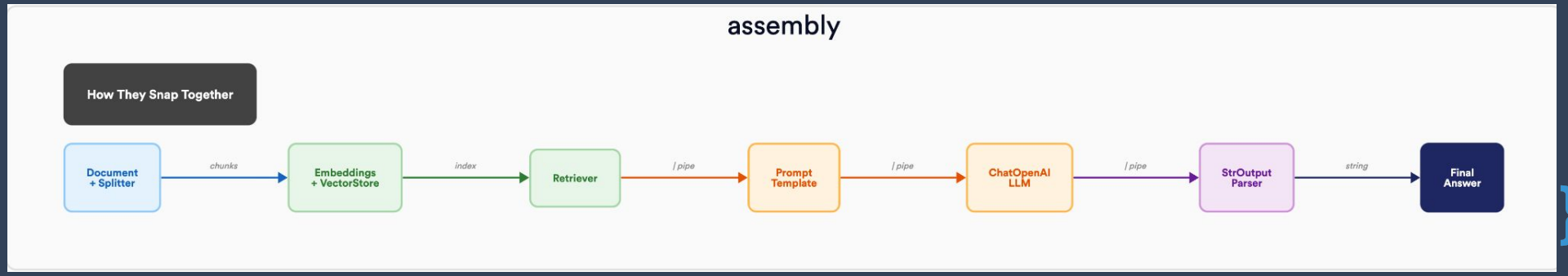
Building the Baseline

{ik} | INTERVIEW KICKSTART

The LangChain Component Stack

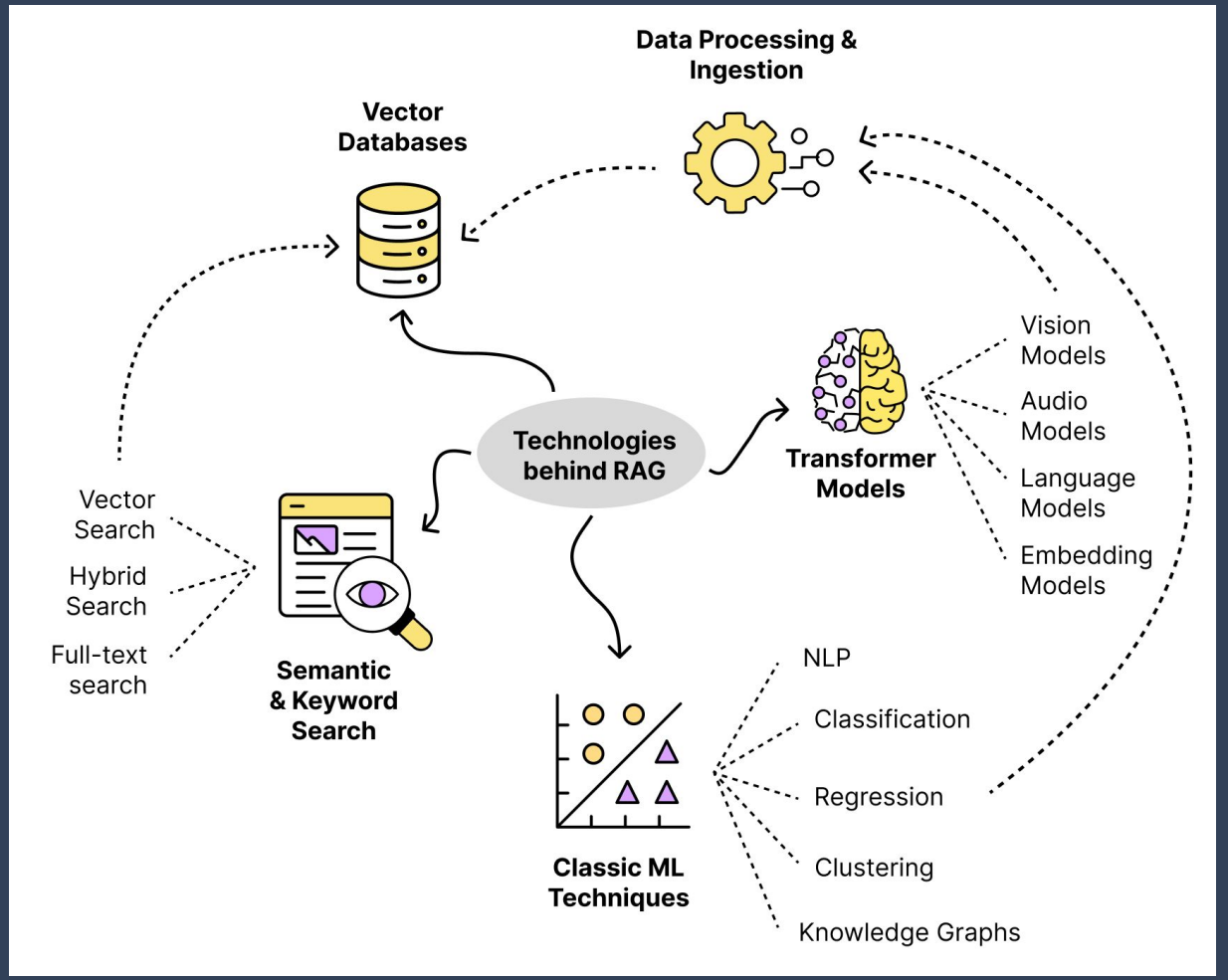


LangChain gives you Lego blocks. LCEL is how you snap them together.



INTERVIEW KICKSTART

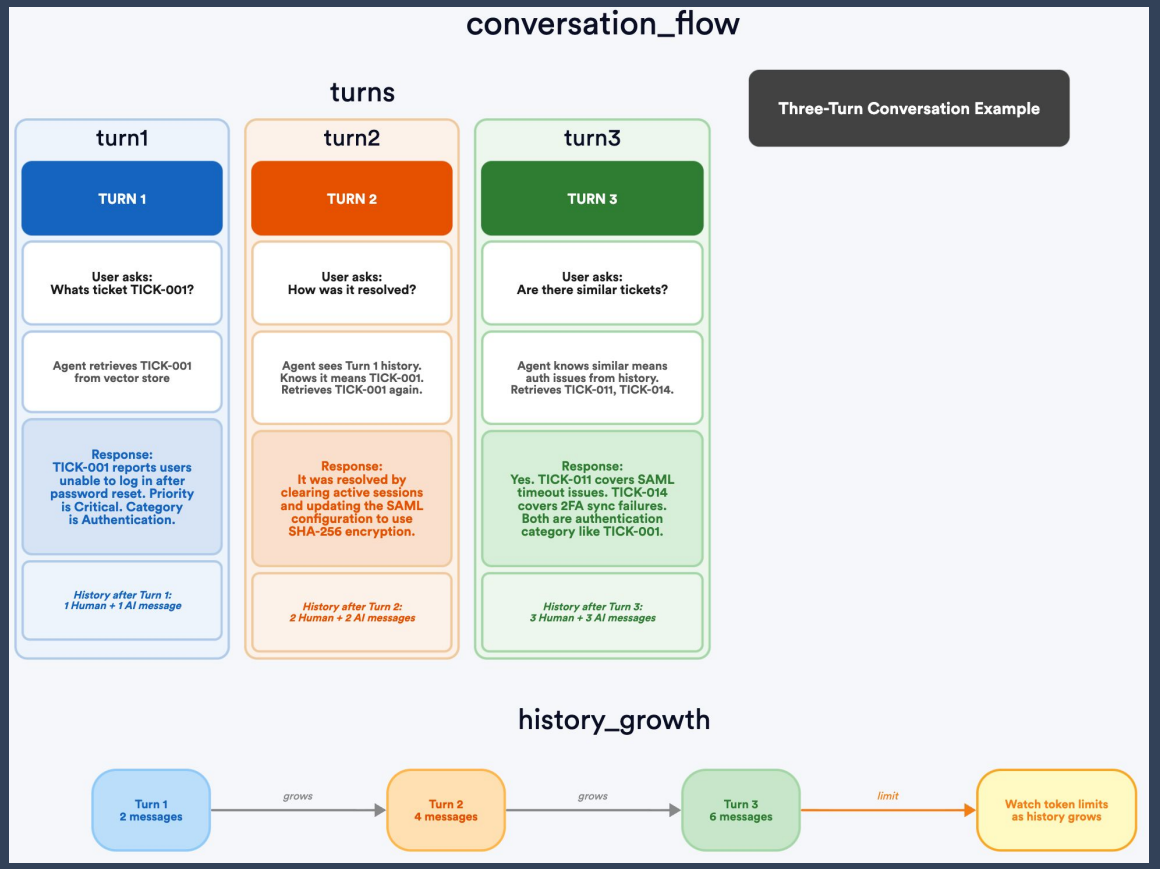
RAG: The Technology Stack Behind the Basics



RAG relies on four core technology pillars, vector databases for storage, semantic/keyword search for retrieval, transformer models (embedding + language) for understanding, and classic ML techniques for processing all orchestrated through a basic data ingestion pipeline, without any advanced query optimization or iterative refinement.

{ik} | INTERVIEW KICKSTART

Conversation with History

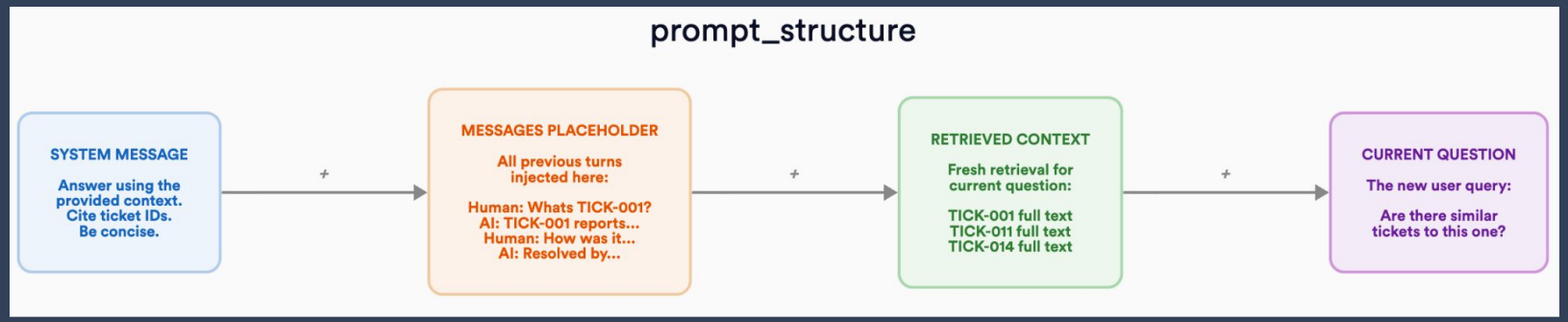




Implementation Pattern

```
chat_history = []

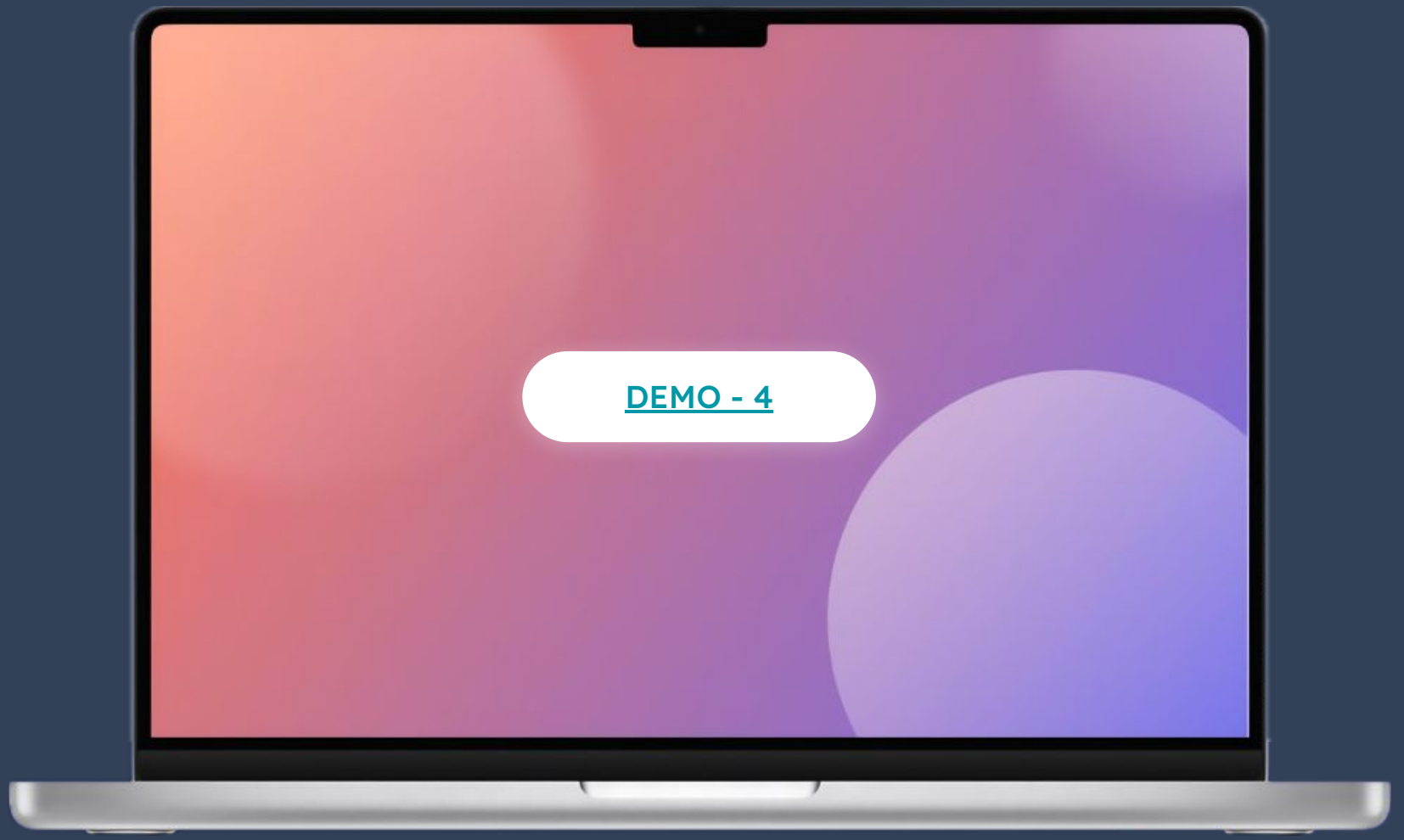
def ask_with_history(question):
context = format_docs(retriever.invoke(question))
response = conv_chain.invoke(
context: context, history: chat_history, question: question
)
chat_history.append(HumanMessage(content=question))
chat_history.append(AIMessage(content=response))
return response
```



Multi-Turn RAG: Remembering Context Across Questions

{ik} INTERVIEW KICKSTART

Module 4 - Implementation Notes: RAG Pipeline



No Question Is Too Small Every Question Deserves an Answer!!

Ask about anything:

- Concepts from today's modules
- How this applies to your work
- Implementation details
- Clarifications on what we covered

Quick Request

- Please do not mind if a question gets skipped accidentally
- Repost politely in chat if needed
- We will try our best to cover as many questions as possible

Remember: If you're wondering about it, others probably are too.



Time for a Break



A 10-minute break is starting now. Please return when the timer ends to continue the presentation

{ik} | INTERVIEW KICKSTART

Today's Agenda

1



RAG Pipeline

2



RAG Evaluation

3



Agentic RAG with LangChain

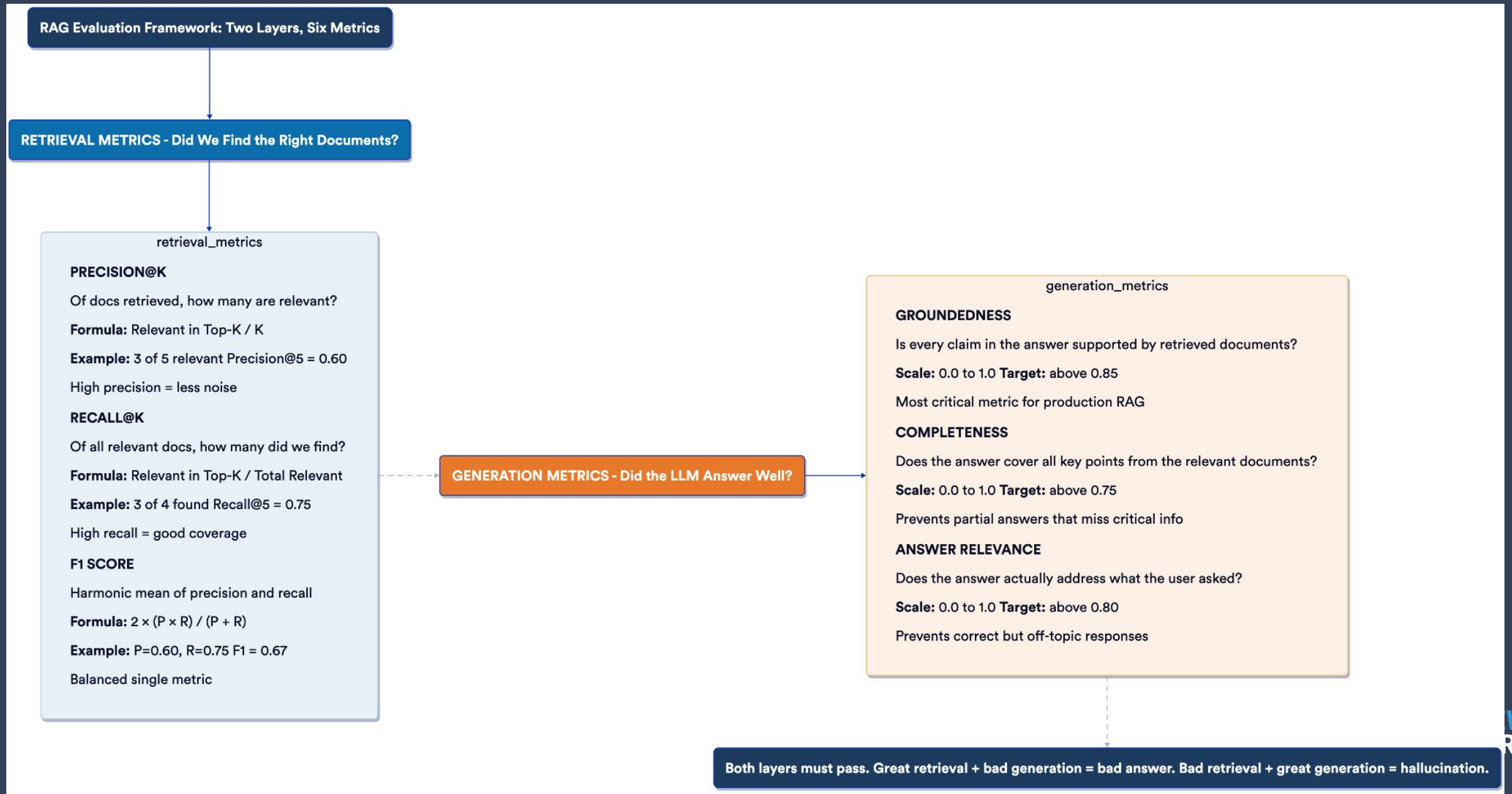
{ik} INTERVIEW KICKSTART

Why Evaluation Matters

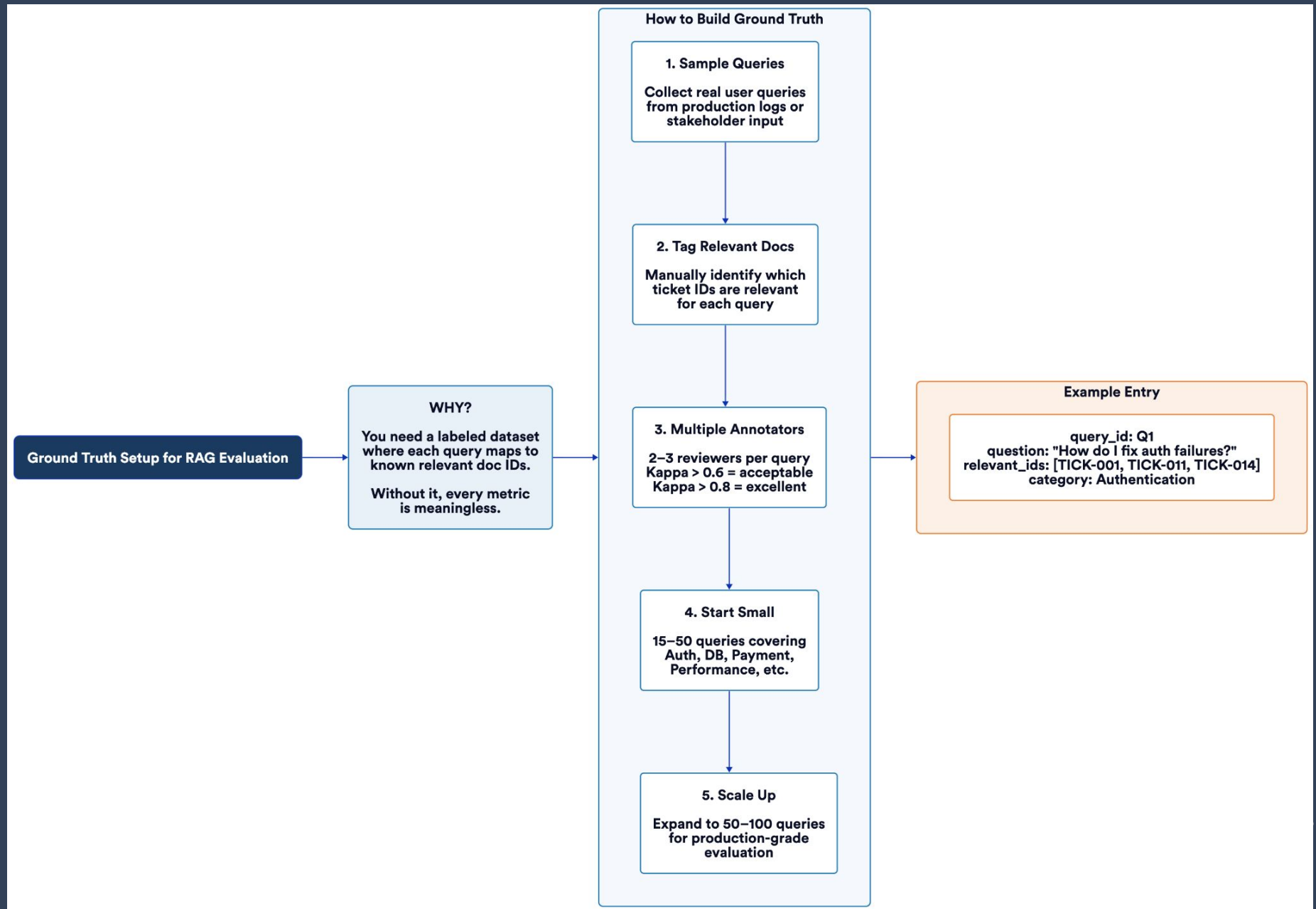
Without evaluation, you are guessing. You cannot tell whether a change to your chunking strategy, embedding model, or prompt template actually improved performance or silently broke something. Evaluation is what transforms "it feels better" into "it is better, and here is the data."

- **Without evaluation:** guesswork, hidden regressions, no way to justify decisions to stakeholders, and the risk of unknowingly deploying broken systems.
- **With evaluation:** data driven decisions, catching regressions before users do, systematic optimization, and the confidence to demonstrate real business value.
- **The core principle:** what gets measured gets improved.

Two-Layer Framework

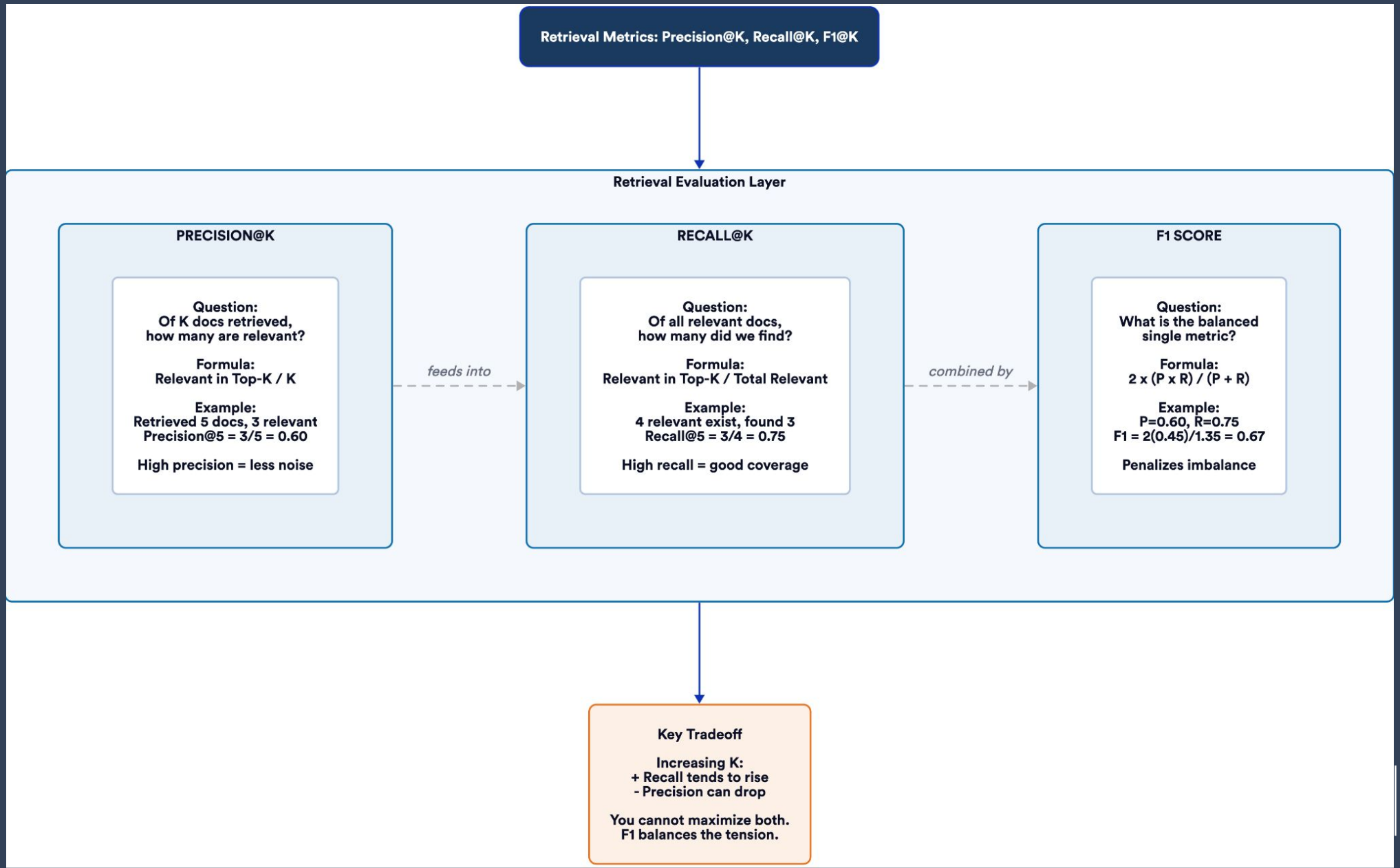


Ground Truth Setup



k} INTERVIEW KICKSTART

Precision@K, Recall@K, F1@K



INTERVIEW KICKSTART

Interpreting K Tradeoffs

The choice of K, how many documents you retrieve, directly shapes the precision recall tradeoff.

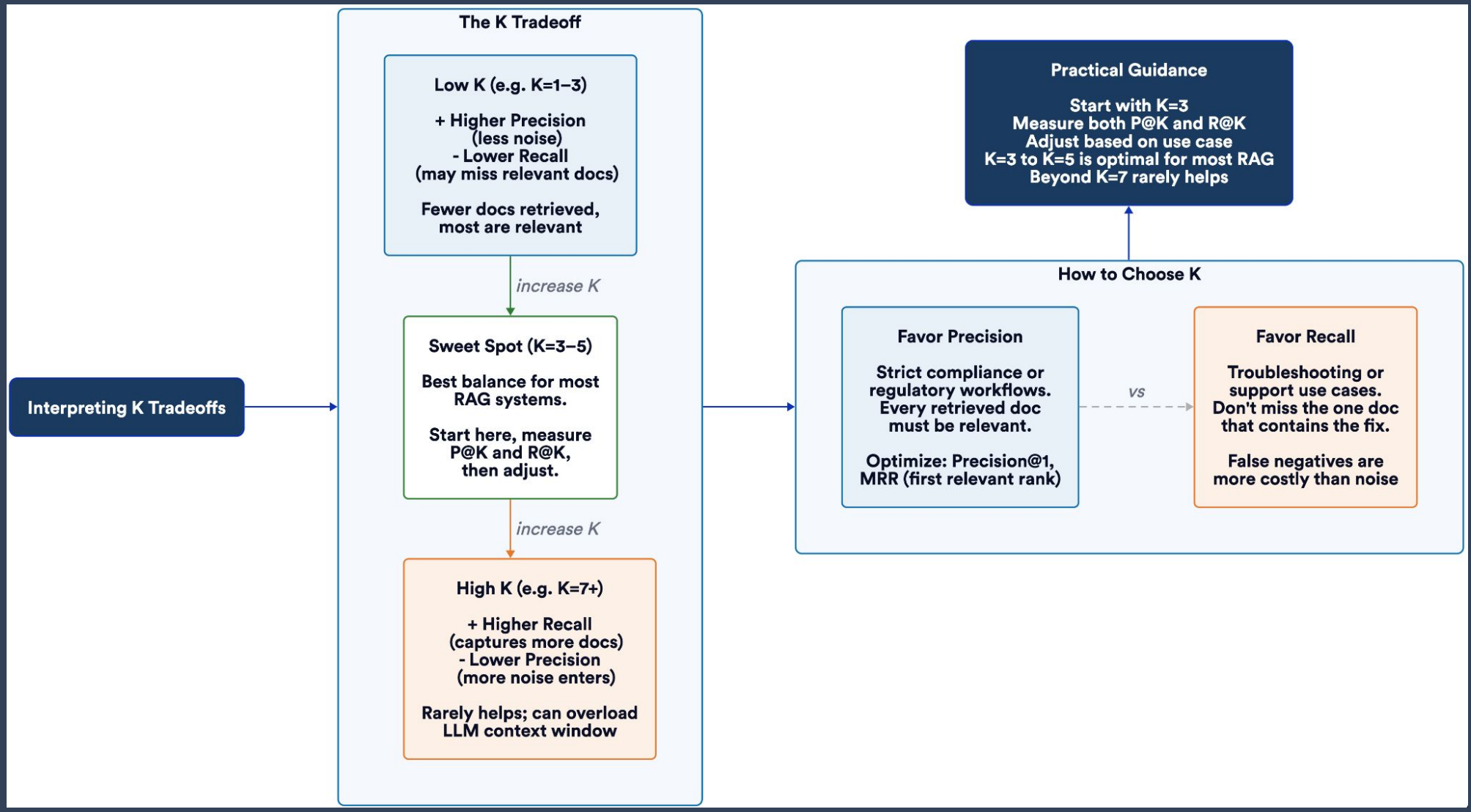
As K increases, recall tends to rise because you have more chances to capture relevant documents. But precision can drop because more irrelevant documents also enter the result set, adding noise.

How to choose K

- **Consider user behavior:** If users only look at the top result, optimize for Precision at 1.
- **Consider risk tolerance:** Are false positives which are irrelevant noise more costly, or are false negatives which are missing documents more costly.
- **Troubleshooting use cases generally favor stronger recall:** You do not want to miss the one document that contains the fix.
- **Strict compliance or regulatory workflows may favor higher precision:** Every retrieved document must be relevant.

{ik} | INTERVIEW KICKSTART

Interpreting K Tradeoffs



{ik} INTERVIEW KICKSTART

Generation Evaluation Metrics

Even with perfect retrieval, generation can fail. This layer evaluates the LLM's answer independently.

Groundedness (Faithfulness): Is every claim in the answer supported by the retrieved context? This is the single most important generation metric for production RAG. A score below 0.85 signals hallucination risk the model is inventing information not present in the documents. Evaluated by having a judge LLM list each claim in the answer and check whether the context supports it.

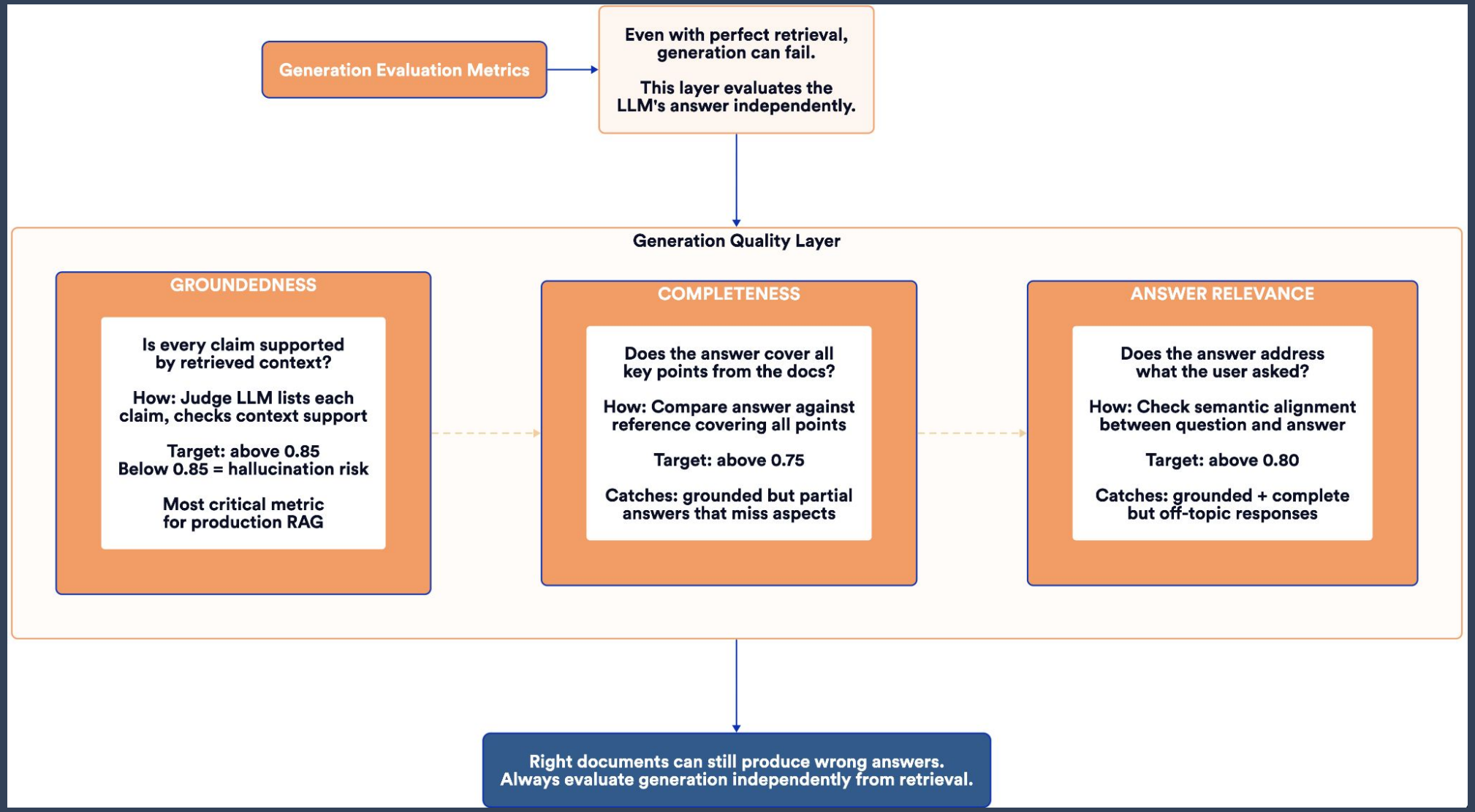
Completeness: Does the answer cover all the key points that the relevant documents contain? A system might be perfectly grounded but only address one of three relevant aspects. Target: above 0.75. Evaluated by comparing the answer against a reference answer that covers all expected points.

Answer Relevance: Does the answer actually address what the user asked? An answer can be grounded and complete but still off-topic if the retrieval pulled tangentially related documents. Target: above 0.80. Evaluated by checking semantic alignment between the question and the answer.

You must evaluate generation even when retrieval scores look strong right documents can still produce wrong answers.

{ik} | INTERVIEW KICKSTART

Generation Evaluation Metrics



{ik} INTERVIEW KICKSTART

LLM-as-Judge

Human evaluation is the gold standard but doesn't scale. LLM-as-Judge bridges this gap by using a powerful model to score answer quality against rubric criteria.

How it works: You send the judge LLM a structured prompt containing the question, the retrieved context, the generated answer, and a scoring rubric. The judge returns a score (0.0–1.0) with a rationale.

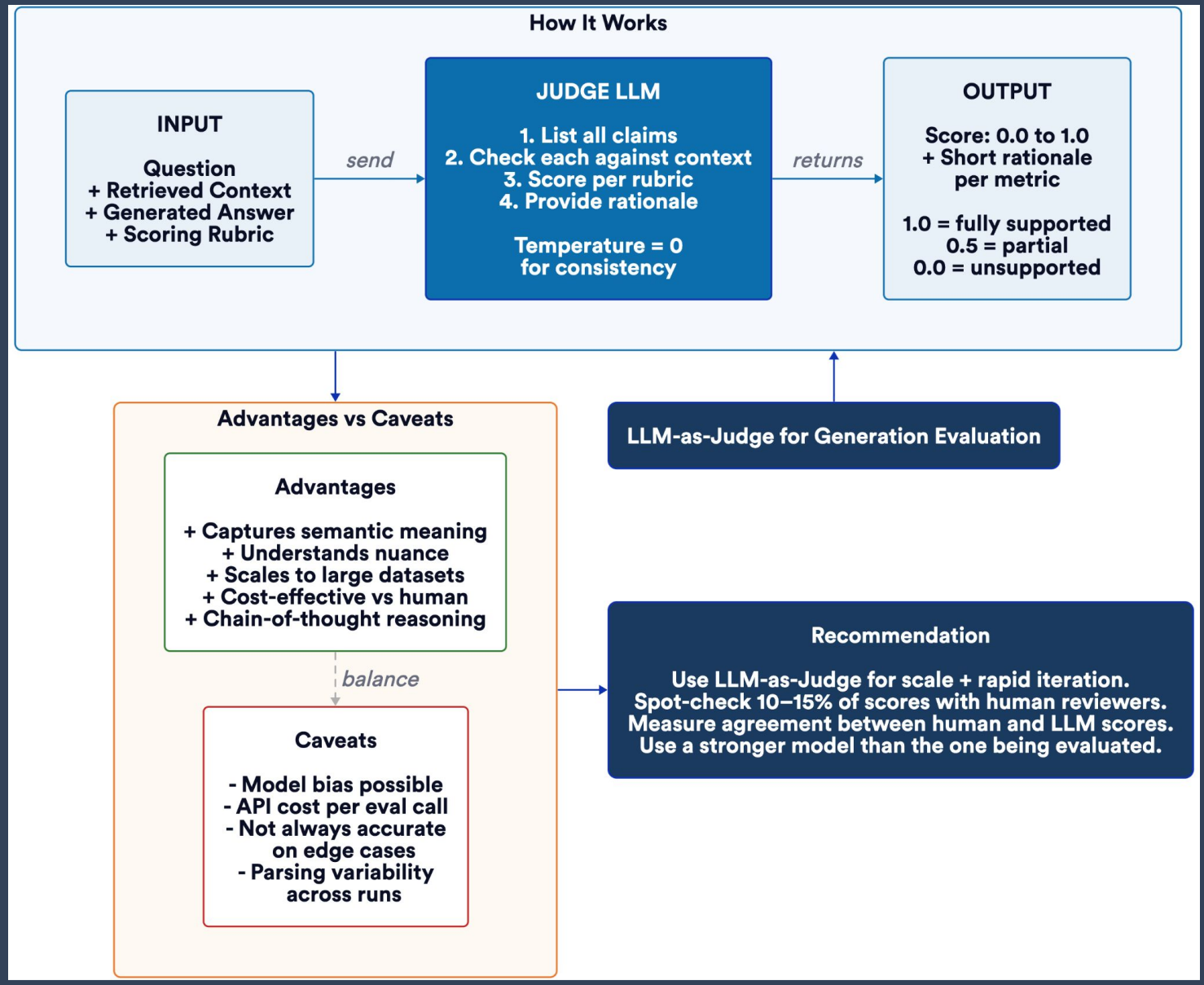
Best practices:

- Use rubric-based prompts with clear scoring anchors (1.0 = fully supported, 0.5 = partially supported, 0.0 = contradicts or unsupported)
- **Ask for chain-of-thought:** list all claims → check each against context → compute score
- Keep temperature at 0 for consistency across runs
- Use a stronger model than the one being evaluated when possible

Caveats to communicate to stakeholders: LLM judges can have model bias, parsing variability across runs, and API cost per evaluation call. They are not always accurate on edge cases.

{ik} | INTERVIEW KICKSTART

LLM-as-Judge



LLM-as-Judge: Automated Evaluation at Scale

{ik} INTERVIEW KICKSTART

Targets & Alert Thresholds

Production systems need clear pass/fail criteria. Here are recommended thresholds:

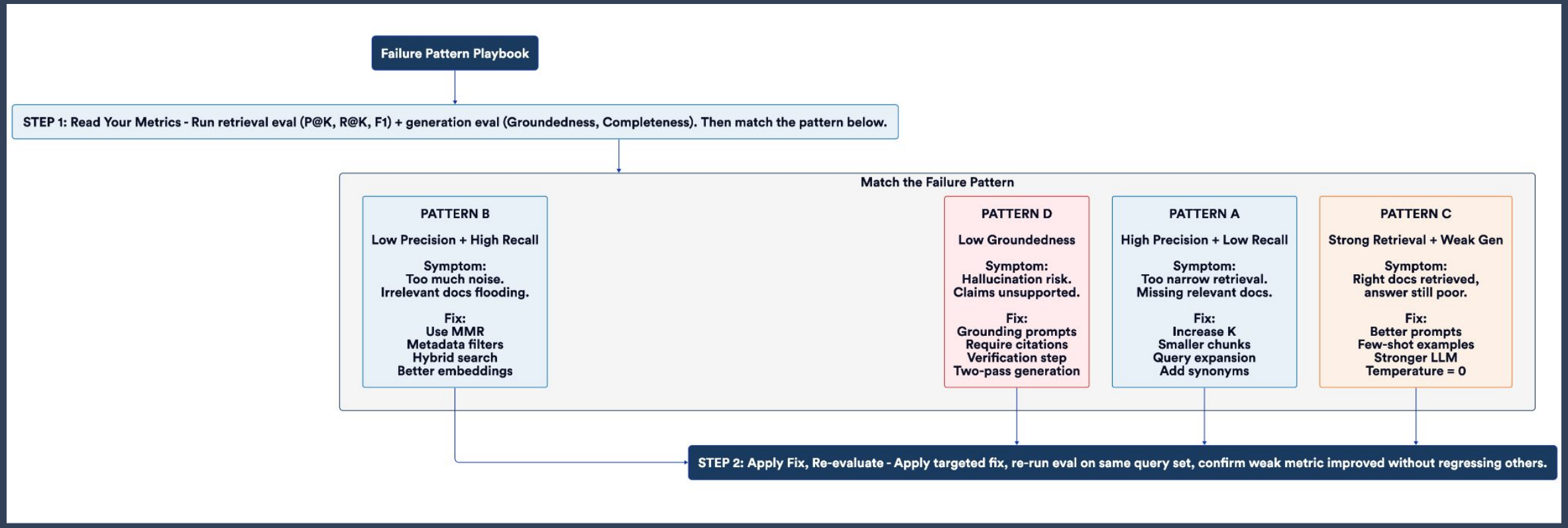
Metric	Target (Green)	Warning (Yellow)	Critical (Red)
Precision@3	> 0.80	< 0.70	< 0.50
Recall@3	> 0.70	< 0.60	< 0.40
F1@3	> 0.75	< 0.65	< 0.45
Groundedness	> 0.85	< 0.75	< 0.60
Completeness	> 0.75	< 0.65	< 0.50

Action rules:

- **Target met:** continue monitoring, log metrics for trend analysis
- **Warning band:** investigate root cause, plan improvement sprint
- **Critical band:** block deployment, escalate immediately the system is producing unreliable outputs

{ik} | INTERVIEW KICKSTART

Failure Pattern Playbook



{ik} INTERVIEW KICKSTART

A/B Testing & Evaluation Pipeline

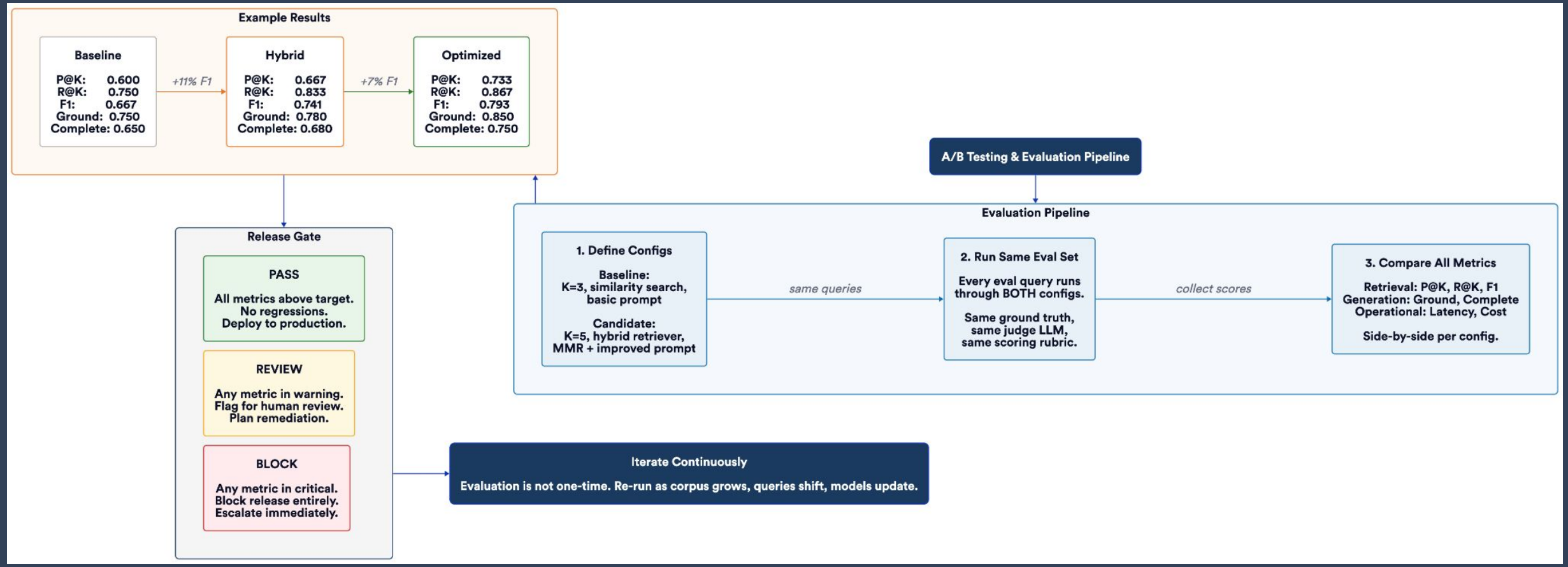
The A/B testing pipeline is how you turn diagnosis into action. Once you've identified a failure pattern (Slide 10), you build a candidate configuration and test it against your baseline using the same evaluation query set.

How it works:

1. Define configurations to compare (e.g., baseline with K=3 similarity search vs. hybrid retriever with MMR and K=5 and an improved prompt)
2. Run every evaluation query through each configuration
3. Compute all six metrics retrieval and generation for each
4. Compare side-by-side: the configuration that lifts the weakest metric without regressing others wins
5. Track latency and cost alongside quality metrics a 5% accuracy gain isn't worth 3x the latency in production

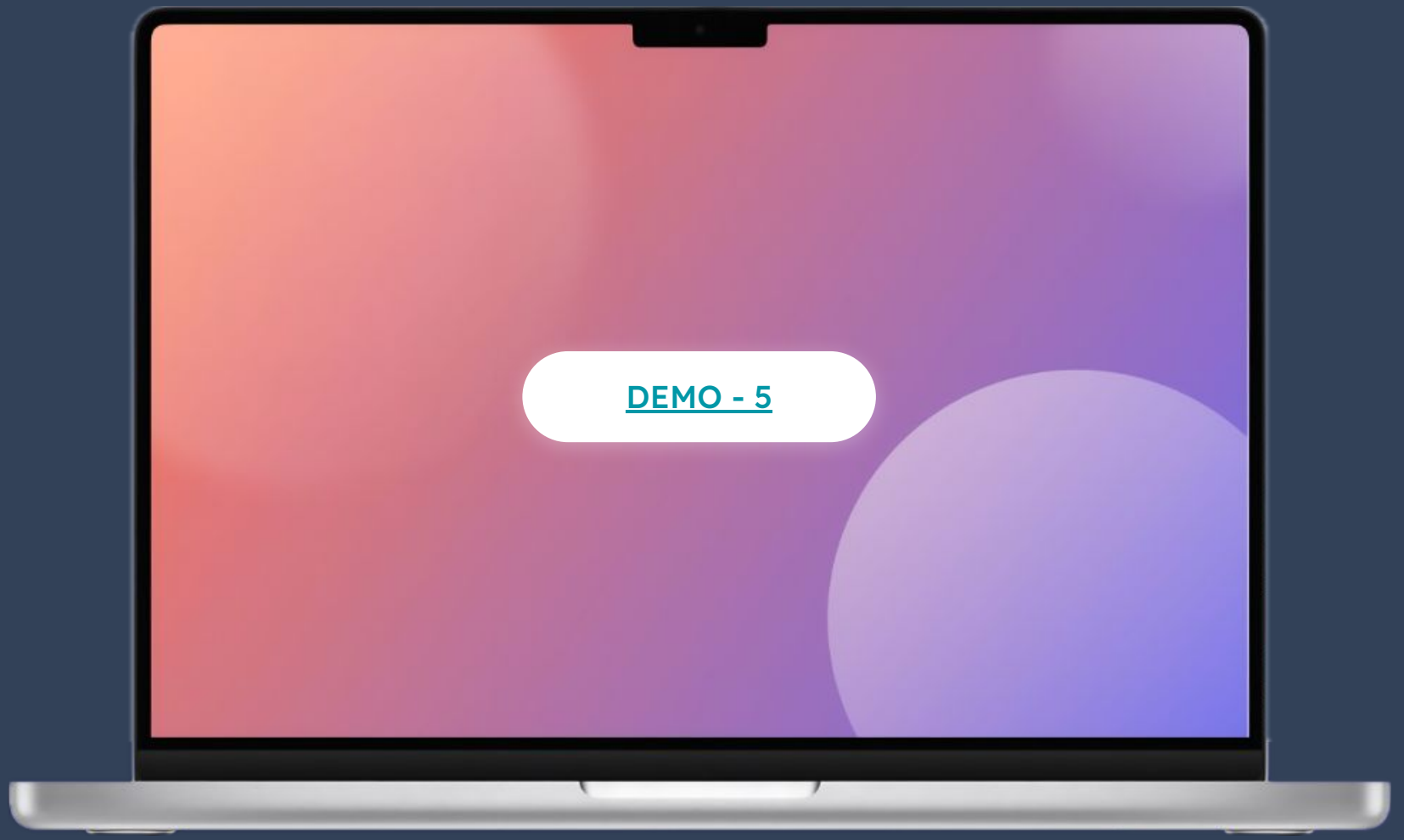
{ik} | INTERVIEW KICKSTART

A/B Testing & Evaluation Pipeline



{ik} INTERVIEW KICKSTART

Module 5 - Implementation Notes: RAG Evaluation



{ik} INTERVIEW KICKSTART

No Question Is Too Small Every Question Deserves an Answer!!

Ask about anything:

- Concepts from today's modules
- How this applies to your work
- Implementation details
- Clarifications on what we covered

Quick Request

- Please do not mind if a question gets skipped accidentally
- Repost politely in chat if needed
- We will try our best to cover as many questions as possible

Remember: If you're wondering about it, others probably are too.



Time for a Break



A 5-minute break is starting now. Please return when the timer ends to continue the presentation

{ik} | INTERVIEW KICKSTART

Today's Agenda

1



RAG Pipeline

2



RAG Evaluation

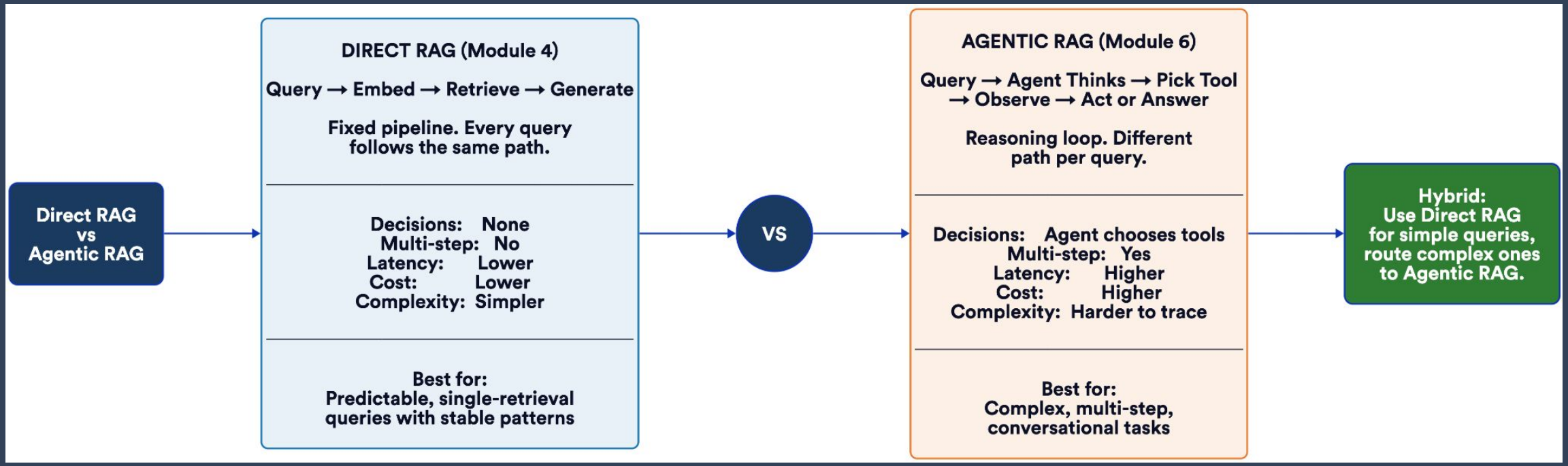
3



Agentic RAG with LangChain

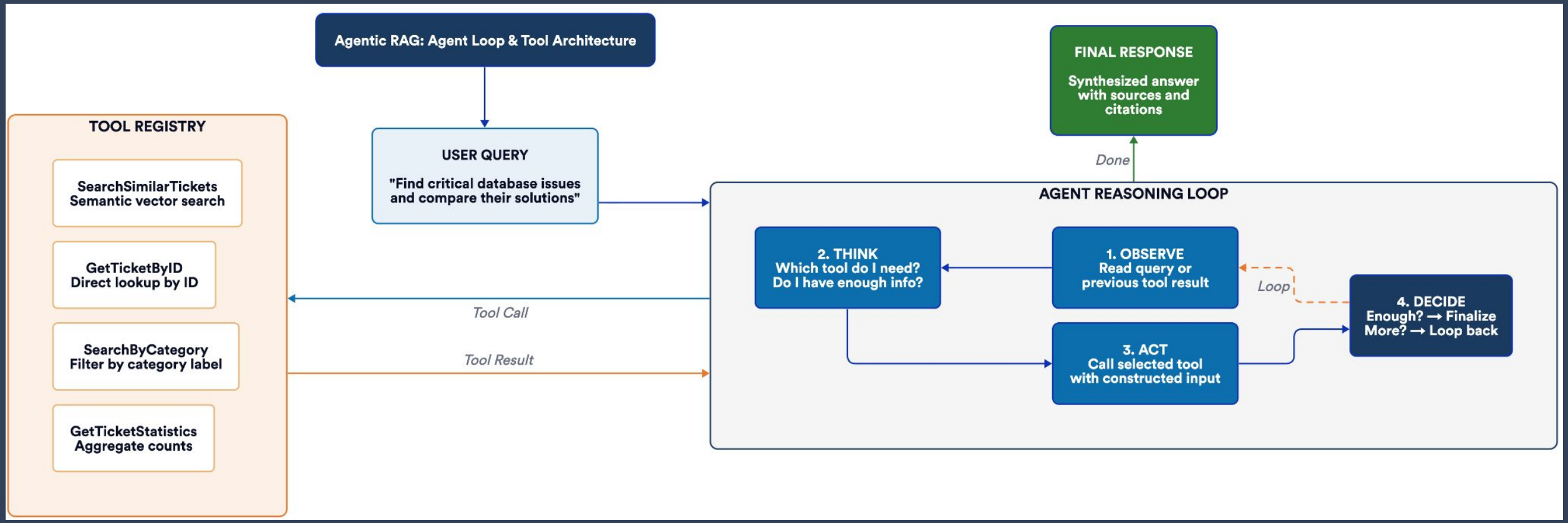
{ik} INTERVIEW KICKSTART

Direct RAG vs Agentic RAG



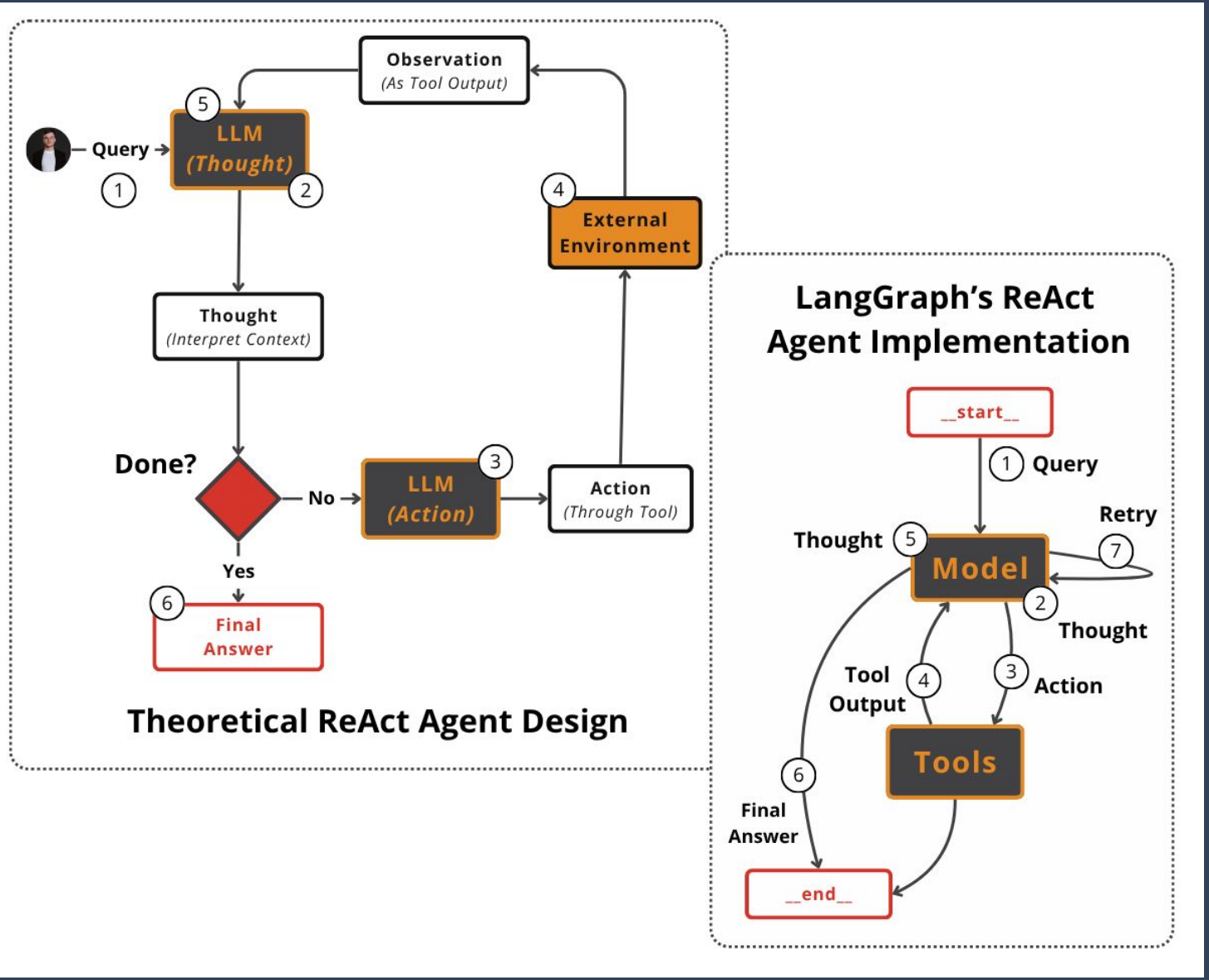
{ik} INTERVIEW KICKSTART

Agent Architecture



{ik} INTERVIEW KICKSTART

The Agent Loop (ReAct Pattern)



The ReAct Loop: Reason → Act → Observe → Repeat

{ik} INTERVIEW KICKSTART

Tool Design Principles

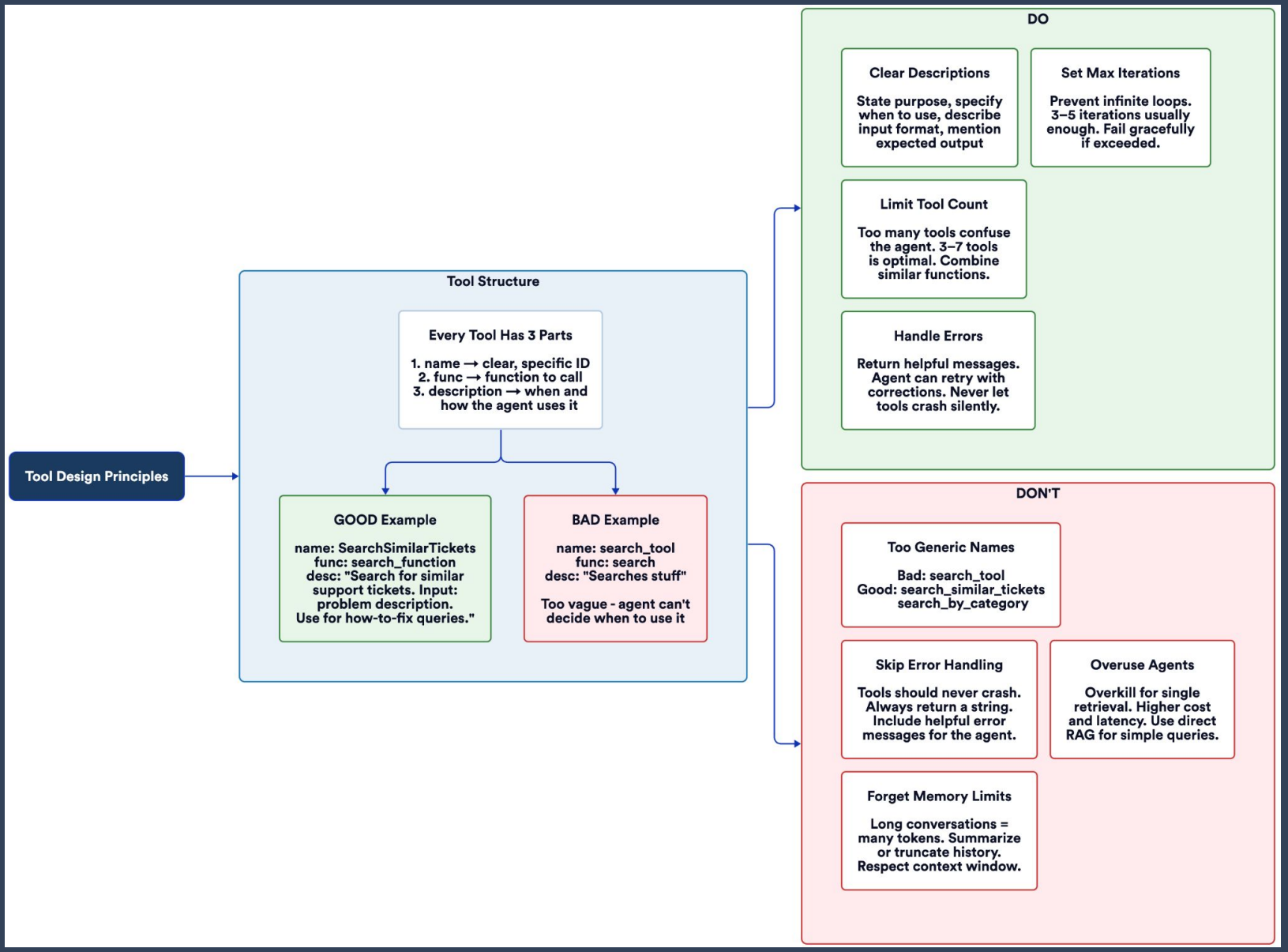
Single-purpose and clearly scoped: Each tool should do one thing well. A tool called search_tool is too vague the agent won't know when to use it versus other options. SearchSimilarTickets tells the agent exactly what it does and when it's appropriate.

Precise descriptions: The tool description is the agent's only guide for selection. It must specify: what the tool does, when to use it, what input format it expects, and what output it returns. Think of the description as a function docstring written for a colleague who has never seen your codebase.

Focused count: Keep the toolset to 3-7 tools. Too few and the agent lacks capability. Too many and tool selection accuracy drops sharply the LLM starts confusing similar tools or picking suboptimal ones. If you need more than 7, consider grouping related functions into a single tool with sub-commands, or routing to specialized sub-agents.

{ik} | INTERVIEW KICKSTART

Tool Design Principles



{ik} INTERVIEW KICKSTART

Example Toolset in Support Desk

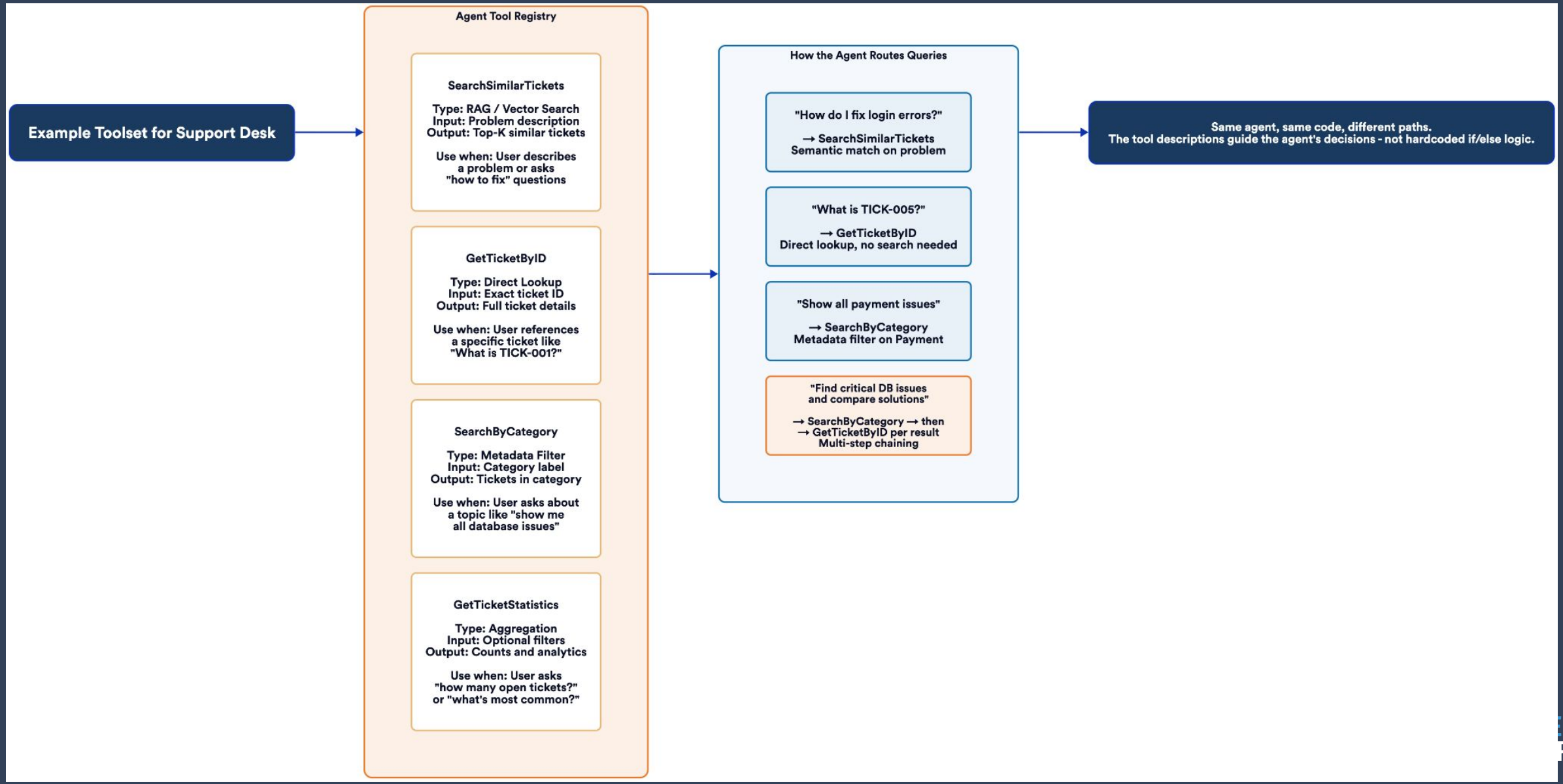
SearchSimilarTickets - Semantic vector search. Takes a problem description as input and returns the top-K most similar tickets by embedding similarity. Use this for "how do I fix..." or "what causes..." style troubleshooting queries.

GetTicketByID - Direct lookup. Takes an exact ticket ID (e.g., TICK-005) and returns full ticket details including description, status, resolution, and timestamps. Use this when the user references a specific ticket.

SearchByCategory - Filtered retrieval. Takes a category label (Authentication, Database, Payment, Performance) and returns all tickets in that category. Use this for scoped exploration queries like "what payment issues exist?"

GetTicketStatistics - Aggregate analytics. Takes an optional category filter and returns counts by status, priority distribution, and resolution rates. Use this for overview and reporting queries.

Example Toolset in Support Desk



Tool Selection Examples

The agent's intelligence shows in how it routes different queries to different tools:

1. **"How do I fix login issues?"** → Agent selects **SearchSimilarTickets**. This is a semantic troubleshooting query - the user is describing a problem, not referencing a specific ticket. Vector similarity search will find the most relevant past resolutions.
2. **"Show me ticket TICK-005"** → Agent selects **GetTicketByID**. The user wants a specific record by its identifier. No semantic search needed - this is a direct lookup.
3. **"What payment issues have we seen?"** → Agent selects **SearchByCategory** with category "Payment". The user wants to explore a topic area, not find a specific document. Category filtering gives a comprehensive scoped result.
4. **"Give me a database overview"** → Agent selects **GetTicketStatistics** with category "Database". The user wants aggregate information - counts, distributions, trends - not individual ticket content.

The descriptions you write for each tool are what make these routing decisions possible. Ambiguous descriptions lead to wrong tool choices.

Multi-Step Reasoning

The real power of agentic RAG emerges when a single query requires multiple tools in sequence. Direct RAG can't do this it's one retrieval, one generation, done.

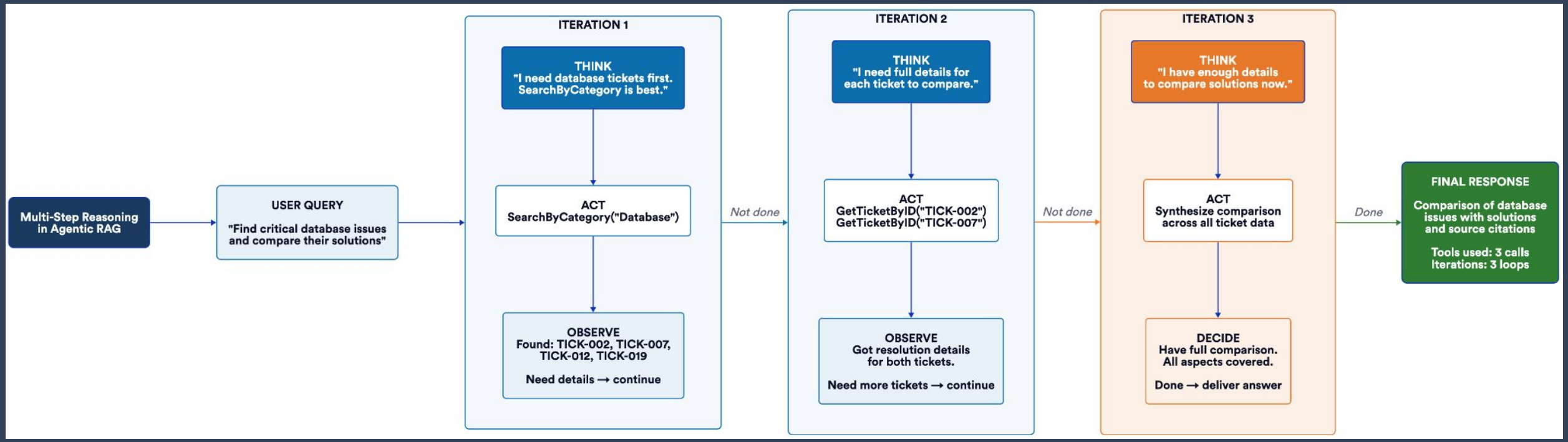
Example query: "Find critical database issues and compare their solutions"

The agent decomposes this into steps:

1. **SearchByCategory("Database")** → returns all database-related tickets
2. **Agent filters results for critical priority** (reasoning step, no tool needed)
3. **GetTicketByID("TICK-002")** → fetches full details for first critical ticket
4. **GetTicketByID("TICK-018")** → fetches full details for second critical ticket
5. **Agent synthesizes a comparison of solutions across both tickets** → delivers final answer

No single tool call could answer the original question. The value of the agentic approach is decomposition (breaking a complex question into sub-tasks) combined with synthesis (assembling partial results into a coherent response).

Multi-Step Reasoning



{ik} INTERVIEW KICKSTART

Best Practices

1. **System prompt clarity:** The agent's system prompt defines its persona, constraints, and reasoning style. Be explicit: tell it to use tools before guessing, to cite sources, to admit when it doesn't know, and to stop when it has enough information.

2. **Tool descriptions are critical:** Spend more time on tool descriptions than on any other part of the agent. A poorly described tool is worse than no tool. It creates confident wrong behavior. Include: what it does, when to use it (and when not to), expected input format, and what the output looks like.

3. **Error handling in every tool:** Tools should never raise unhandled exceptions. Return clear error strings the agent can reason about: "No ticket found with ID TICK-999" is actionable. A Python traceback is not.

4. **Set max iterations:** Always cap the agent loop at 3-5 iterations. If the agent hasn't answered by then, it's likely stuck. Have it return a graceful fallback: "I wasn't able to fully answer your question. Here's what I found so far..."

5. **Monitor costs:** Agentic RAG uses more tokens than direct RAG because every reasoning step and tool call consumes input/output tokens. Track per-query token usage and latency. Consider using a cheaper model for agent reasoning (e.g., GPT-4o-mini) and a stronger model only for final synthesis.

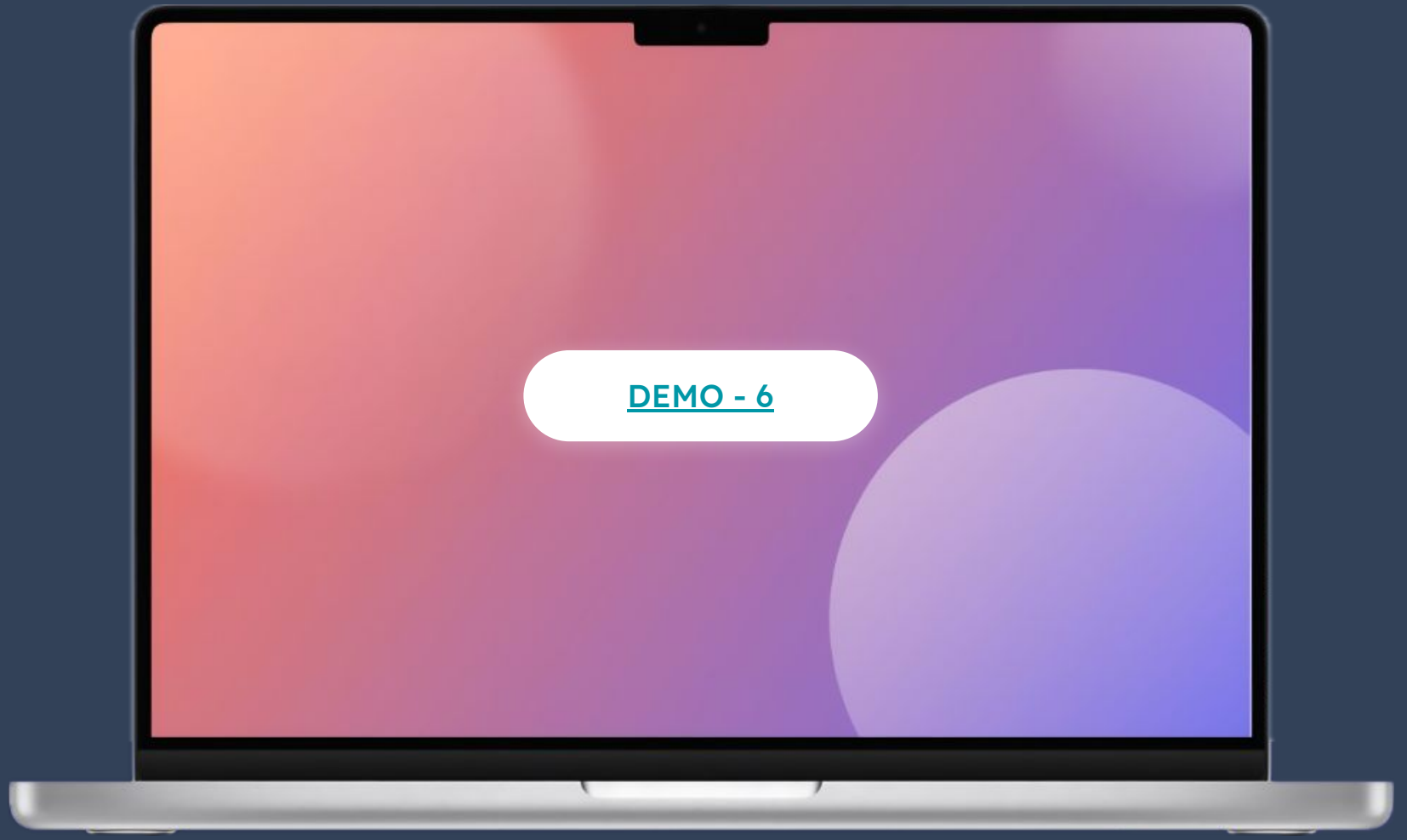
Live Zoom Poll



We are launching a poll on Zoom. Please take 2 minutes to fill out the form.

{ik} | INTERVIEW KICKSTART

Module 6 - Implementation Notes: Agentic RAG with LangChain





{ik} | INTERVIEW KICKSTART